

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

- └─ index.html (73766 B / afbadb0adfd3)
- └─ main.js (33823 B / 04a8e344941e)
- └─ style.css (14083 B / 3bb92984b4f4)

==== index.html (73766 B / afbadb0adfd3) ====

```
1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>TS Chess</title>
7 |     <script id="ts-chess-shared-core-source" type="text/plain">
8 |       var TSChessShared = (function (exports) {
9 |         'use strict';
10 |         const MASK64 = 0xfffffffffffffn;
11 |         function rotl(x, k) {
12 |           return 0xfffffffffffffn & ((x << k) | (x >> (64n - k)));
13 |         }
14 |         function wrappingMul(x, y) {
15 |           return (x * y) & MASK64;
16 |         }
17 |         function xoroshiro128(state) {
18 |           return function () {
19 |             let s0 = BigInt(state & MASK64),
20 |                 s1 = BigInt((state >> 64n) & MASK64);
21 |             const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
22 |             return (
23 |               (s1 ^ s0),
24 |               (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
25 |               (s1 = rotl(s1, 37n)),
26 |               (state = (s1 << 64n) | s0),
27 |               result
28 |             );
29 |           };
30 |         }
31 |         const rand = xoroshiro128(0xa187eb39cdcaed8f31c4b365b102e01en),
32 |               PIECE_KEYS = Array.from({ length: 2 }, () =>
33 |                 Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
34 |               EP_KEYS = Array.from({ length: 8 }, () => rand()),
35 |               CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
36 |               SIDE_KEY = rand(),
37 |               DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1';
38 |         class Move {
39 |           color;
40 |           from;
41 |           to;
42 |           piece;
43 |           captured;
44 |           promotion;
45 |           flags;
46 |           san;
47 |           lan;
48 |           before;
49 |           after;
50 |           constructor(chess, internal) {
51 |             const {
52 |               color: color,
53 |               piece: piece,
54 |               from: from,
55 |               to: to,
56 |               flags: flags,
57 |               captured: captured,
58 |               promotion: promotion,
59 |             } = internal,
60 |               fromAlgebraic = algebraic(from),
```

```

62 |         .toAlgebraic = algebraic(to);
63 |         ((this.color = color),
64 |         (this.piece = piece),
65 |         (this.from = fromAlgebraic),
66 |         (this.to = toAlgebraic),
67 |         (this.san = chess.moveToSan(internal, chess.moves({ legal: !0 }))),
68 |         (this.lan = fromAlgebraic + toAlgebraic),
69 |         (this.before = chess.fen()),
70 |         chess._makeMove(internal),
71 |         (this.after = chess.fen()),
72 |         chess._undoMove(),
73 |         (this.flags = ''));
74 |         for (const flag in BITS) BITS[flag] & flags & (this.flags += FLAGS[flag]);
75 |         (captured & (this.captured = captured),
76 |         promotion & ((this.promotion = promotion), (this.lan += promotion)));
77 |     }
78 |     isCapture() {
79 |         return this.flags.indexOf(FLAGS.CAPTURE) > -1;
80 |     }
81 |     isPromotion() {
82 |         return this.flags.indexOf(FLAGS.PROMOTION) > -1;
83 |     }
84 |     isEnPassant() {
85 |         return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
86 |     }
87 |     isKingsideCastle() {
88 |         return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
89 |     }
90 |     isQueensideCastle() {
91 |         return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
92 |     }
93 |     isBigPawn() {
94 |         return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
95 |     }
96 | }
97 | const FLAGS = {
98 |     NORMAL: 'n',
99 |     CAPTURE: 'c',
100 |     BIG_PAWN: 'b',
101 |     EP_CAPTURE: 'e',
102 |     PROMOTION: 'p',
103 |     KSIDE_CASTLE: 'k',
104 |     QSIDE_CASTLE: 'q',
105 |     NULL_MOVE: '-',
106 | },
107 | BITS = {
108 |     NORMAL: 1,
109 |     CAPTURE: 2,
110 |     BIG_PAWN: 4,
111 |     EP_CAPTURE: 8,
112 |     PROMOTION: 16,
113 |     KSIDE_CASTLE: 32,
114 |     QSIDE_CASTLE: 64,
115 |     NULL_MOVE: 128,
116 | },
117 | 0x88 = {
118 |     a8: 0,
119 |     b8: 1,
120 |     c8: 2,
121 |     d8: 3,
122 |     e8: 4,
123 |     f8: 5,
124 |     g8: 6,
125 |     h8: 7,
126 |     a7: 16,
127 |     b7: 17,
128 |     c7: 18,
129 |     d7: 19,
130 |     e7: 20,
131 |     f7: 21,
132 |     g7: 22,

```

```

133 | . | . | . | . | h7: 23,
134 | . | . | . | . | a6: 32,
135 | . | . | . | . | b6: 33,
136 | . | . | . | . | c6: 34,
137 | . | . | . | . | d6: 35,
138 | . | . | . | . | e6: 36,
139 | . | . | . | . | f6: 37,
140 | . | . | . | . | g6: 38,
141 | . | . | . | . | h6: 39,
142 | . | . | . | . | a5: 48,
143 | . | . | . | . | b5: 49,
144 | . | . | . | . | c5: 50,
145 | . | . | . | . | d5: 51,
146 | . | . | . | . | e5: 52,
147 | . | . | . | . | f5: 53,
148 | . | . | . | . | g5: 54,
149 | . | . | . | . | h5: 55,
150 | . | . | . | . | a4: 64,
151 | . | . | . | . | b4: 65,
152 | . | . | . | . | c4: 66,
153 | . | . | . | . | d4: 67,
154 | . | . | . | . | e4: 68,
155 | . | . | . | . | f4: 69,
156 | . | . | . | . | g4: 70,
157 | . | . | . | . | h4: 71,
158 | . | . | . | . | a3: 80,
159 | . | . | . | . | b3: 81,
160 | . | . | . | . | c3: 82,
161 | . | . | . | . | d3: 83,
162 | . | . | . | . | e3: 84,
163 | . | . | . | . | f3: 85,
164 | . | . | . | . | g3: 86,
165 | . | . | . | . | h3: 87,
166 | . | . | . | . | a2: 96,
167 | . | . | . | . | b2: 97,
168 | . | . | . | . | c2: 98,
169 | . | . | . | . | d2: 99,
170 | . | . | . | . | e2: 100,
171 | . | . | . | . | f2: 101,
172 | . | . | . | . | g2: 102,
173 | . | . | . | . | h2: 103,
174 | . | . | . | . | a1: 112,
175 | . | . | . | . | b1: 113,
176 | . | . | . | . | c1: 114,
177 | . | . | . | . | d1: 115,
178 | . | . | . | . | e1: 116,
179 | . | . | . | . | f1: 117,
180 | . | . | . | . | g1: 118,
181 | . | . | . | . | h1: 119,
182 | . | . | . | . | },
183 | . | . | . | . | PAWN_OFFSETS = {-b: [16, 32, 17, 15], w: [-16, -32, -17, -15]},
184 | . | . | . | . | PIECE_OFFSETS = {
185 | . | . | . | . | n: [-18, -33, -31, -14, 18, 33, 31, 14],
186 | . | . | . | . | b: [-17, -15, 17, 15],
187 | . | . | . | . | r: [-16, 1, 16, -1],
188 | . | . | . | . | q: [-17, -16, -15, 1, 17, 16, 15, -1],
189 | . | . | . | . | k: [-17, -16, -15, 1, 17, 16, 15, -1],
190 | . | . | . | . | },
191 | . | . | . | . | ATTACKS = [
192 | . | . | . | . | 20, 0, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0,
193 | . | . | . | . | 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0,
194 | . | . | . | . | 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2,
195 | . | . | . | . | 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24,
196 | . | . | . | . | 24, 24, 24, 56, 0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0,
197 | . | . | . | . | 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20,
198 | . | . | . | . | 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0,
199 | . | . | . | . | 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0,
200 | . | . | . | . | 24, 0, 0, 0, 0, 0, 0, 20,
201 | . | . | . | . | ],
202 | . | . | . | . | RAYS = [
203 | . | . | . | . | 17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0,

```

```

204 | . . . . . 15, 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0,
205 | . . . . . 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0,
206 | . . . . . 16, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
207 | . . . . . 1, 1, 0, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0,
208 | . . . . . 0, 0, 0, 0, 0, -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17,
209 | . . . . . 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0,
210 | . . . . . -16, 0, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0,
211 | . . . . . 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, 0, 0, -17,
212 | . . . . . ],
213 | . . . . . PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
214 | . . . . . PROMOTIONS = [ 'n', 'b', 'r', 'q' ],
215 | . . . . . ROOKS = {
216 | . . . . . w: [
217 | . . . . . { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
218 | . . . . . { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
219 | . . . . . ],
220 | . . . . . b: [
221 | . . . . . { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
222 | . . . . . { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
223 | . . . . . ],
224 | . . . . . },
225 | . . . . . SECOND_RANK = { b: 1, w: 6 };
226 | . . . . . function rank(square) {
227 | . . . . . return square >> 4;
228 | . . . . . }
229 | . . . . . function file(square) {
230 | . . . . . return 15 & square;
231 | . . . . . }
232 | . . . . . function isDigit(c) {
233 | . . . . . return -1 !== '0123456789'.indexOf(c);
234 | . . . . . }
235 | . . . . . function algebraic(square) {
236 | . . . . . const f = file(square),
237 | . . . . . r = rank(square);
238 | . . . . . return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
239 | . . . . . }
240 | . . . . . function swapColor(color) {
241 | . . . . . return 'w' === color ? 'b' : 'w';
242 | . . . . . }
243 | . . . . . function validateFen(fen) {
244 | . . . . . const tokens = fen.split(/\s+/);
245 | . . . . . if (6 !== tokens.length)
246 | . . . . . return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
247 | . . . . . const moveNumber = parseInt(tokens[5], 10);
248 | . . . . . if (isNaN(moveNumber) || moveNumber <= 0)
249 | . . . . . return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
250 | . . . . . const halfMoves = parseInt(tokens[4], 10);
251 | . . . . . if (isNaN(halfMoves) || halfMoves < 0)
252 | . . . . . return {
253 | . . . . . ok: !1,
254 | . . . . . error: 'Invalid FEN: half move counter number must be a non-negative integer',
255 | . . . . . };
256 | . . . . . if (!/^(?![abcdefgh][36])$/.test(tokens[3]))
257 | . . . . . return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
258 | . . . . . if (/^[^kqQ-]/.test(tokens[2]))
259 | . . . . . return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
260 | . . . . . if (!/^(w|b)$/.test(tokens[1]))
261 | . . . . . return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
262 | . . . . . const rows = tokens[0].split('/');
263 | . . . . . if (8 !== rows.length)
264 | . . . . . return { ok: !1, error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows' };
265 | . . . . . for (let i = 0; i < rows.length; i++) {
266 | . . . . . let sumFields = 0,
267 | . . . . . previousWasNumber = !1;
268 | . . . . . for (let k = 0; k < rows[i].length; k++)
269 | . . . . . if (isDigit(rows[i][k])) {
270 | . . . . . if (previousWasNumber)
271 | . . . . . return { ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)' };
272 | . . . . . ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
273 | . . . . . } else {
274 | . . . . . if (!/^[prnbqkPRNBQK]$/.test(rows[i][k]))

```

```

275 |         return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
276 |         ((sumFields += 1), (previousWasNumber = !1));
277 |     }
278 |     if (8 !== sumFields)
279 |         return { ok: !1, error: 'Invalid FEN: piece data is invalid (too many squares in rank)' };
280 |     }
281 |     if (('3' === tokens[3][1] & 'w' === tokens[1]) || ('6' === tokens[3][1] & 'b' === tokens[1]))
282 |         return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
283 |     const kings = [
284 |         { color: 'white', regex: /K/g },
285 |         { color: 'black', regex: /k/g },
286 |     ];
287 |     for (const { color: color, regex: regex } of kings) {
288 |         if (!regex.test(tokens[0])) return { ok: !1, error: `Invalid FEN: missing ${color} king` };
289 |         if ((tokens[0].match(regex) || []).length > 1)
290 |             return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
291 |     }
292 |     return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
293 |         ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
294 |         : { ok: !0 };
295 | }
296 | function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
297 |     const r = rank(to);
298 |     if ('p' !== piece || (7 !== r & 0 !== r))
299 |         moves.push({
300 |             color: color,
301 |             from: from,
302 |             to: to,
303 |             piece: piece,
304 |             captured: captured,
305 |             flags: flags,
306 |         });
307 |     else
308 |         for (let i = 0; i < PROMOTIONS.length; i++) {
309 |             const promotion = PROMOTIONS[i];
310 |             moves.push({
311 |                 color: color,
312 |                 from: from,
313 |                 to: to,
314 |                 piece: piece,
315 |                 captured: captured,
316 |                 promotion: promotion,
317 |                 flags: flags | BITS.PROMOTION,
318 |             });
319 |         }
320 | }
321 | return (
322 |     (exports.BISHOP = 'b'),
323 |     (exports.BLACK = 'b'),
324 |     (exports.Chess = class Chess {
325 |         _board = new Array(128);
326 |         _turn = 'w';
327 |         _kings = { w: -1, b: -1 };
328 |         _epSquare = -1;
329 |         _halfMoves = 0;
330 |         _moveNumber = 0;
331 |         _history = [];
332 |         _castling = { w: 0, b: 0 };
333 |         _hash = 0n;
334 |         _positionCount = new Map();
335 |         constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
336 |             this.load(fen, { skipValidation: skipValidation });
337 |         }
338 |         clear() {
339 |             ((this._board = new Array(128)),
340 |             (this._kings = { w: -1, b: -1 }),
341 |             (this._turn = 'w'),
342 |             (this._castling = { w: 0, b: 0 }),
343 |             (this._epSquare = -1),
344 |             (this._halfMoves = 0),
345 |             (this._moveNumber = 1),

```

```

346 |         (this._history = []),
347 |         (this._hash = this._computeHash()),
348 |         (this._positionCount = new Map());
349 |     }
350 |     load(fen, { skipValidation: skipValidation !== 1 } = {}) {
351 |         let tokens = fen.split(/\s+/);
352 |         if (tokens.length ≥ 2 && tokens.length < 6) {
353 |             const adjustments = ['-', '-', '0', '1'];
354 |             fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
355 |         }
356 |         if (((tokens = fen.split(/\s+/)), !skipValidation)) {
357 |             const { ok, error } = validateFen(fen);
358 |             if (!ok) throw new Error(error);
359 |         }
360 |         const position = tokens[0];
361 |         let square = 0;
362 |         this.clear();
363 |         for (let i = 0; i < position.length; i++) {
364 |             const piece = position.charAt(i);
365 |             if ('/ ' === piece) square += 8;
366 |             else if (isDigit(piece)) square += parseInt(piece, 10);
367 |             else {
368 |                 const color = piece < 'a' ? 'w' : 'b';
369 |                 (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
370 |             }
371 |         }
372 |         ((this._turn = tokens[1],
373 |         tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
374 |         tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
375 |         tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
376 |         tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
377 |         (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
378 |         (this._halfMoves = parseInt(tokens[4], 10)),
379 |         (this._moveNumber = parseInt(tokens[5], 10)),
380 |         (this._hash = this._computeHash()),
381 |         this._incPositionCount());
382 |         }
383 |         fen({ forceEnpassantSquare: forceEnpassantSquare !== 1 } = {}) {
384 |             let empty = 0,
385 |                 fen = '';
386 |             for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
387 |                 if (this._board[i]) {
388 |                     empty > 0 && ((fen += empty), (empty = 0));
389 |                     const { color, type: piece } = this._board[i];
390 |                     fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
391 |                 } else empty++;
392 |                 (i + 1) & 136 &&
393 |                 (empty > 0 && (fen += empty), i ≡ 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
394 |             }
395 |             let castling = '';
396 |             (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
397 |             this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
398 |             this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
399 |             this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
400 |             (castling = castling || '-'));
401 |             let epSquare = '-';
402 |             if (-1 ≡ this._epSquare)
403 |                 if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
404 |             else {
405 |                 const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
406 |                     squares = [bigPawnSquare + 1, bigPawnSquare - 1];
407 |                 for (const square of squares) {
408 |                     if (136 & square) continue;
409 |                     const color = this._turn;
410 |                     if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
411 |                         this._makeMove({
412 |                             color: color,
413 |                             from: square,
414 |                             to: this._epSquare,
415 |                             piece: 'p',
416 |                             captured: 'p',

```

```

417 |         .flags: BITS.EP_CAPTURE,
418 |     });
419 |     const isLegal = !this._isKingAttacked(color);
420 |     if ((this._undoMove(), isLegal)) {
421 |         epSquare = algebraic(this._epSquare);
422 |         break;
423 |     }
424 | }
425 | }
426 | }
427 | return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
428 | }
429 | _pieceKey(i) {
430 |     if (!this._board[i]) return 0n;
431 |     const { color, type } = this._board[i],
432 |           typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
433 |     return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
434 | }
435 | _epKey() {
436 |     return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
437 | }
438 | _castlingKey() {
439 |     const index = (this._castling.w >> 5) | (this._castling.b >> 3);
440 |     return CASTLING_KEYS[index];
441 | }
442 | _computeHash() {
443 |     let hash = 0n;
444 |     for (let i = 0x88.a8; i <= 0x88.h1; i++)
445 |         136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
446 |     return (
447 |         (hash ^= this._epKey()),
448 |         (hash ^= this._castlingKey()),
449 |         'b' === this._turn && (hash ^= SIDE_KEY),
450 |         hash
451 |     );
452 | }
453 | reset() {
454 |     this.load(DEFAULT_POSITION);
455 | }
456 | get(square) {
457 |     return this._board[0x88[square]];
458 | }
459 | _set(sq, piece) {
460 |     ((this._hash ^= this._pieceKey(sq)),
461 |     (this._board[sq] = piece),
462 |     (this._hash ^= this._pieceKey(sq)));
463 | }
464 | put({ type, color }, square) {
465 |     if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
466 |     if (!(square in 0x88)) return !1;
467 |     const sq = 0x88[square];
468 |     if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
469 |     const currentPieceOnSquare = this._board[sq];
470 |     return (
471 |         currentPieceOnSquare &&
472 |         'k' === currentPieceOnSquare.type &&
473 |         (this._kings[currentPieceOnSquare.color] !== -1),
474 |         this._set(sq, { type, color }),
475 |         'k' === type && (this._kings[color] = sq),
476 |         !0
477 |     );
478 | }
479 | _clear(sq) {
480 |     ((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
481 | }
482 | attacked(color, square, verbose) {
483 |     const attackers = [];
484 |     for (let i = 0x88.a8; i <= 0x88.h1; i++) {
485 |         if (136 & i) {
486 |             i += 7;
487 |             continue;

```

```

488 |         }
489 |         if (void 0 === this._board[i] || this._board[i].color !== color) continue;
490 |         const piece = this._board[i],
491 |               difference = i - square;
492 |         if (0 === difference) continue;
493 |         const index = difference + 119;
494 |         if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
495 |             if ('p' === piece.type) {
496 |                 if (
497 |                     (difference > 0 & 'w' === piece.color) ||
498 |                     (difference < 0 & 'b' === piece.color)
499 |                 ) {
500 |                     if (!verbose) return !0;
501 |                     attackers.push(algebraic(i));
502 |                 }
503 |                 continue;
504 |             }
505 |             if ('n' === piece.type || 'k' === piece.type) {
506 |                 if (verbose) {
507 |                     attackers.push(algebraic(i));
508 |                     continue;
509 |                 }
510 |                 return !0;
511 |             }
512 |             const offset = RAYS[index];
513 |             let j = i + offset,
514 |                 blocked = !1;
515 |             for (; j !== square; ) {
516 |                 if (null !== this._board[j]) {
517 |                     blocked = !0;
518 |                     break;
519 |                 }
520 |                 j += offset;
521 |             }
522 |             if (!blocked) {
523 |                 if (verbose) {
524 |                     attackers.push(algebraic(i));
525 |                     continue;
526 |                 }
527 |                 return !0;
528 |             }
529 |         }
530 |     }
531 |     return !!verbose & attackers;
532 | }
533 | attackers(square, attackedBy) {
534 |     return attackedBy
535 |         ? this._attacked(attackedBy, 0x88[square], !0)
536 |         : this._attacked(this._turn, 0x88[square], !0);
537 | }
538 | _isKingAttacked(color) {
539 |     const square = this._kings[color];
540 |     return -1 !== square & this._attacked(swapColor(color), square);
541 | }
542 | hash() {
543 |     return this._hash.toString(16);
544 | }
545 | isAttacked(square, attackedBy) {
546 |     return this._attacked(attackedBy, 0x88[square]);
547 | }
548 | isCheck() {
549 |     return this._isKingAttacked(this._turn);
550 | }
551 | inCheck() {
552 |     return this.isCheck();
553 | }
554 | isCheckmate() {
555 |     return this.isCheck() & 0 === this._moves().length;
556 | }
557 | isStalemate() {
558 |     return !this.isCheck() & 0 === this._moves().length;

```



```

559 |         }
560 |         isInsufficientMaterial() {
561 |             const pieces = { b: 0, n: 0, r: 0, q: 0, k: 0, p: 0 },
562 |                   bishops = [];
563 |             let numPieces = 0,
564 |                 squareColor = 0;
565 |             for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
566 |                 if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
567 |                     i += 7;
568 |                     continue;
569 |                 }
570 |                 const piece = this._board[i];
571 |                 piece &&
572 |                     ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
573 |                     'b' === piece.type && bishops.push(squareColor),
574 |                     numPieces++);
575 |             }
576 |             if (2 === numPieces) return !0;
577 |             if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
578 |             if (numPieces === pieces.b + 2) {
579 |                 let sum = 0;
580 |                 const len = bishops.length;
581 |                 for (let i = 0; i < len; i++) sum += bishops[i];
582 |                 if (0 === sum || sum === len) return !0;
583 |             }
584 |             return !1;
585 |         }
586 |         isThreefoldRepetition() {
587 |             return this._getPositionCount(this._hash) ≥ 3;
588 |         }
589 |         isDrawByFiftyMoves() {
590 |             return this._halfMoves ≥ 100;
591 |         }
592 |         isDraw() {
593 |             return (
594 |                 this.isDrawByFiftyMoves() ||
595 |                 this.isStalemate() ||
596 |                 this.isInsufficientMaterial() ||
597 |                 this.isThreefoldRepetition()
598 |             );
599 |         }
600 |         isGameOver() {
601 |             return this.isCheckmate() || this.isDraw();
602 |         }
603 |         moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
604 |             const moves = this._moves({ square: square, piece: piece });
605 |             return verbose
606 |                 ? moves.map((move) => new Move(this, move))
607 |                 : moves.map((move) => this._moveToSan(move, moves));
608 |         }
609 |         _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
610 |             const forSquare = square ? square.toLowerCase() : void 0,
611 |                   forPiece = piece ? piece.toLowerCase(),
612 |                   moves = [],
613 |                   us = this._turn,
614 |                   them = swapColor(us);
615 |             let firstSquare = 0x88.a8,
616 |                 lastSquare = 0x88.h1,
617 |                 singleSquare = !1;
618 |             if (forSquare) {
619 |                 if (!(forSquare in 0x88)) return [];
620 |                 ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
621 |             }
622 |             for (let from = firstSquare; from ≤ lastSquare; from++) {
623 |                 if (136 & from) {
624 |                     from += 7;
625 |                     continue;
626 |                 }
627 |                 if (!this._board[from] || this._board[from].color === them) continue;
628 |                 const { type: type } = this._board[from];
629 |                 let to;

```

```

630 |         if ('p' === type) {
631 |             if (forPiece && forPiece !== type) continue;
632 |             ((to = from + PAWN_OFFSETS[us][0]),
633 |             this._board[to] ||
634 |             (addMove(moves, us, from, to, 'p'),
635 |             (to = from + PAWN_OFFSETS[us][1]),
636 |             SECOND_RANK[us] !== rank(from) ||
637 |             this._board[to] ||
638 |             addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN)));
639 |             for (let j = 2; j < 4; j++)
640 |                 ((to = from + PAWN_OFFSETS[us][j]),
641 |                 136 & to ||
642 |                 (this._board[to]?.color === them
643 |                 ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
644 |                 : to === this._epSquare &&
645 |                 addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE)));
646 |         } else {
647 |             if (forPiece && forPiece !== type) continue;
648 |             for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
649 |                 const offset = PIECE_OFFSETS[type][j];
650 |                 for (to = from; (to += offset), !(136 & to);) {
651 |                     if (this._board[to]) {
652 |                         if (this._board[to].color === us) break;
653 |                         addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
654 |                         break;
655 |                     }
656 |                     if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
657 |                 }
658 |             }
659 |         }
660 |     }
661 |     if (
662 |         !(
663 |             (void 0 !== forPiece && 'k' !== forPiece) ||
664 |             (singleSquare && lastSquare !== this._kings[us])
665 |         )
666 |     ) {
667 |         if (this._castling[us] & BITS.KSIDE_CASTLE) {
668 |             const castlingFrom = this._kings[us],
669 |                 castlingTo = castlingFrom + 2;
670 |             this._board[castlingFrom + 1] ||
671 |             this._board[castlingTo] ||
672 |             this._attacked(them, this._kings[us]) ||
673 |             this._attacked(them, castlingFrom + 1) ||
674 |             this._attacked(them, castlingTo) ||
675 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
676 |         }
677 |         if (this._castling[us] & BITS.QSIDE_CASTLE) {
678 |             const castlingFrom = this._kings[us],
679 |                 castlingTo = castlingFrom - 2;
680 |             this._board[castlingFrom - 1] ||
681 |             this._board[castlingFrom - 2] ||
682 |             this._board[castlingFrom - 3] ||
683 |             this._attacked(them, this._kings[us]) ||
684 |             this._attacked(them, castlingFrom - 1) ||
685 |             this._attacked(them, castlingTo) ||
686 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
687 |         }
688 |     }
689 |     if (!legal || -1 === this._kings[us]) return moves;
690 |     const legalMoves = [];
691 |     for (let i = 0, len = moves.length; i < len; i++)
692 |         (this._makeMove(moves[i]),
693 |         this._isKingAttacked(us) || legalMoves.push(moves[i]),
694 |         this._undoMove());
695 |     return legalMoves;
696 | }
697 | move(move) {
698 |     let moveObj = null;
699 |     const moves = this._moves();
700 |     for (let i = 0, len = moves.length; i < len; i++)

```

```

701 |         if (
702 |             move.from === algebraic(moves[i].from) &&
703 |             move.to === algebraic(moves[i].to) &&
704 |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
705 |         ) {
706 |             moveObj = moves[i];
707 |             break;
708 |         }
709 |         if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
710 |         const prettyMove = new Move(this, moveObj);
711 |         return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
712 |     }
713 |     _push(move) {
714 |         this._history.push({
715 |             move: move,
716 |             kings: { b: this._kings.b, w: this._kings.w },
717 |             turn: this._turn,
718 |             castling: { b: this._castling.b, w: this._castling.w },
719 |             epSquare: this._epSquare,
720 |             halfMoves: this._halfMoves,
721 |             moveNumber: this._moveNumber,
722 |         });
723 |     }
724 |     _movePiece(from, to) {
725 |         ((this._hash ^= this._pieceKey(from)),
726 |         (this._board[to] = this._board[from]),
727 |         delete this._board[from],
728 |         (this._hash ^= this._pieceKey(to)));
729 |     }
730 |     _makeMove(move) {
731 |         const us = this._turn,
732 |             them = swapColor(us);
733 |         if ((this._push(move), move.flags & BITS.NULL_MOVE))
734 |             return (
735 |                 'b' === us && this._moveNumber++,
736 |                 this._halfMoves++,
737 |                 (this._turn = them),
738 |                 void (this._epSquare = -1)
739 |             );
740 |         if (
741 |             ((this._hash ^= this._epKey()),
742 |             (this._hash ^= this._castlingKey()),
743 |             move.captured && (this._hash ^= this._pieceKey(move.to)),
744 |             this._movePiece(move.from, move.to),
745 |             move.flags & BITS.EP_CAPTURE &&
746 |             ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
747 |             move.promotion &&
748 |             (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
749 |             'k' === this._board[move.to].type)
750 |         ) {
751 |             if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
752 |                 const castlingTo = move.to - 1,
753 |                     castlingFrom = move.to + 1;
754 |                 this._movePiece(castlingFrom, castlingTo);
755 |             } else if (move.flags & BITS.QSIDE_CASTLE) {
756 |                 const castlingTo = move.to + 1,
757 |                     castlingFrom = move.to - 2;
758 |                 this._movePiece(castlingFrom, castlingTo);
759 |             }
760 |             this._castling[us] = 0;
761 |         }
762 |         if (this._castling[us])
763 |             for (let i = 0, len = ROOKS[us].length; i < len; i++)
764 |                 if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
765 |                     this._castling[us] ^= ROOKS[us][i].flag;
766 |                     break;
767 |                 }
768 |         if (this._castling[them])
769 |             for (let i = 0, len = ROOKS[them].length; i < len; i++)
770 |                 if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
771 |                     this._castling[them] ^= ROOKS[them][i].flag;

```

```

772 |         break;
773 |     }
774 |     if (((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
775 |         let epSquare;
776 |         ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
777 |          ((move.to - 1) & 136 ||
778 |           'p' === this._board[move.to - 1] ? type :
779 |           this._board[move.to - 1] ? color : them) &
780 |          ((move.to + 1) & 136 ||
781 |           'p' === this._board[move.to + 1] ? type :
782 |           this._board[move.to + 1] ? color : them)
783 |          ? (this._epSquare = -1)
784 |          : ((this._epSquare = epSquare), (this._hash ^= this._epKey())));
785 |     } else this._epSquare = -1;
786 |     ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
787 |      ? (this._halfMoves = 0)
788 |      : this._halfMoves++,
789 |      'b' === us && this._moveNumber++,
790 |      (this._turn = them),
791 |      (this._hash ^= SIDE_KEY));
792 | }
793 | undo() {
794 |     const hash = this._hash,
795 |           move = this._undoMove();
796 |     if (move) {
797 |         const prettyMove = new Move(this, move);
798 |         return (this._decPositionCount(hash), prettyMove);
799 |     }
800 |     return null;
801 | }
802 | _undoMove() {
803 |     const old = this._history.pop();
804 |     if (void 0 === old) return null;
805 |     ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
806 |     const move = old.move;
807 |     ((this._kings = old.kings),
808 |      (this._turn = old.turn),
809 |      (this._castling = old.castling),
810 |      (this._epSquare = old.epSquare),
811 |      (this._halfMoves = old.halfMoves),
812 |      (this._moveNumber = old.moveNumber),
813 |      (this._hash ^= this._epKey()),
814 |      (this._hash ^= this._castlingKey()),
815 |      (this._hash ^= SIDE_KEY));
816 |     const us = this._turn,
817 |           them = swapColor(us);
818 |     if (move.flags & BITS.NULL_MOVE) return move;
819 |     if (
820 |         (this._movePiece(move.to, move.from),
821 |          move.piece &&
822 |          (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
823 |          move.captured)
824 |         )
825 |         if (move.flags & BITS.EP_CAPTURE) {
826 |             let index;
827 |             ((index = 'b' === us ? move.to - 16 : move.to + 16),
828 |              this._set(index, { type: 'p', color: them }));
829 |         } else this._set(move.to, { type: move.captured, color: them });
830 |     if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
831 |         let castlingTo, castlingFrom;
832 |         (move.flags & BITS.KSIDE_CASTLE
833 |          ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
834 |          : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
835 |          this._movePiece(castlingFrom, castlingTo));
836 |     }
837 |     return move;
838 | }
839 | _moveToSan(move, moves) {
840 |     let output = '';
841 |     if (move.flags & BITS.KSIDE_CASTLE) output = 'O-O';
842 |     else if (move.flags & BITS.QSIDE_CASTLE) output = 'O-O-O';

```

```

843 |         else {
844 |             if ('p' !== move.piece) {
845 |                 const disambiguator = (function getDisambiguator(move, moves) {
846 |                     const from = move.from,
847 |                           to = move.to,
848 |                           piece = move.piece;
849 |                     let ambiguities = 0,
850 |                         sameRank = 0,
851 |                         sameFile = 0;
852 |                     for (let i = 0, len = moves.length; i < len; i++) {
853 |                         const ambigFrom = moves[i].from,
854 |                               ambigTo = moves[i].to;
855 |                         piece === moves[i].piece &&
856 |                         from !== ambigFrom &&
857 |                         to === ambigTo &&
858 |                         (ambiguities++,
859 |                          rank(from) === rank(ambigFrom) && sameRank++,
860 |                          file(from) === file(ambigFrom) && sameFile++);
861 |                     }
862 |                     return ambiguities > 0
863 |                         ? sameRank > 0 && sameFile > 0
864 |                         ? algebraic(from)
865 |                         : sameFile > 0
866 |                         ? algebraic(from).charAt(1)
867 |                         : algebraic(from).charAt(0)
868 |                         : '';
869 |                 })(move, moves);
870 |                 output += move.piece.toUpperCase() + disambiguator;
871 |             }
872 |             (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
873 |              ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
874 |              (output += algebraic(move.to)),
875 |              move.promotion && (output += '=' + move.promotion.toUpperCase()));
876 |         }
877 |         return (
878 |             this._makeMove(move),
879 |             this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
880 |             this._undoMove(),
881 |             output
882 |         );
883 |     }
884 |     turn() {
885 |         return this._turn;
886 |     }
887 |     board() {
888 |         const output = [];
889 |         let row = [];
890 |         for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
891 |             (null === this._board[i]
892 |              ? row.push(null)
893 |              : row.push({
894 |                  square: algebraic(i),
895 |                  type: this._board[i].type,
896 |                  color: this._board[i].color,
897 |              })),
898 |             (i += 1) & 136 && (output.push(row), (row = []), (i += 8));
899 |         return output;
900 |     }
901 |     squareColor(square) {
902 |         if (square in 0x88) {
903 |             const sq = 0x88[square];
904 |             return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
905 |         }
906 |         return null;
907 |     }
908 |     history({ verbose: verbose = !1 } = {}) {
909 |         const reversedHistory = [],
910 |             moveHistory = [];
911 |         for (; this._history.length > 0;) reversedHistory.push(this._undoMove());
912 |         for (;;) {
913 |             const move = reversedHistory.pop();

```

```

914 |         if (!move) break;
915 |         (verbose
916 |         ? moveHistory.push(new Move(this, move))
917 |         : moveHistory.push(this._moveToSan(move, this._moves()))),
918 |         this._makeMove(move));
919 |     }
920 |     return moveHistory;
921 | }
922 | _getPositionCount(hash) {
923 |     return this._positionCount.get(hash) ?? 0;
924 | }
925 | _incPositionCount() {
926 |     this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
927 | }
928 | _decPositionCount(hash) {
929 |     const currentCount = this._positionCount.get(hash) ?? 0;
930 |     1 === currentCount
931 |     ? this._positionCount.delete(hash)
932 |     : this._positionCount.set(hash, currentCount - 1);
933 | }
934 | moveNumber() {
935 |     return this._moveNumber;
936 | }
937 | })),
938 | (exports.DEFAULT_POSITION = DEFAULT_POSITION),
939 | (exports.KING = 'k'),
940 | (exports.KNIGHT = 'n'),
941 | (exports.Move = Move),
942 | (exports.PAWN = 'p'),
943 | (exports.QUEEN = 'q'),
944 | (exports.ROOK = 'r'),
945 | (exports.SQUARES = [
946 |     'a8',
947 |     'b8',
948 |     'c8',
949 |     'd8',
950 |     'e8',
951 |     'f8',
952 |     'g8',
953 |     'h8',
954 |     'a7',
955 |     'b7',
956 |     'c7',
957 |     'd7',
958 |     'e7',
959 |     'f7',
960 |     'g7',
961 |     'h7',
962 |     'a6',
963 |     'b6',
964 |     'c6',
965 |     'd6',
966 |     'e6',
967 |     'f6',
968 |     'g6',
969 |     'h6',
970 |     'a5',
971 |     'b5',
972 |     'c5',
973 |     'd5',
974 |     'e5',
975 |     'f5',
976 |     'g5',
977 |     'h5',
978 |     'a4',
979 |     'b4',
980 |     'c4',
981 |     'd4',
982 |     'e4',
983 |     'f4',
984 |     'g4',

```

```

985 |         'h4',
986 |         'a3',
987 |         'b3',
988 |         'c3',
989 |         'd3',
990 |         'e3',
991 |         'f3',
992 |         'g3',
993 |         'h3',
994 |         'a2',
995 |         'b2',
996 |         'c2',
997 |         'd2',
998 |         'e2',
999 |         'f2',
1000 |         'g2',
1001 |         'h2',
1002 |         'a1',
1003 |         'b1',
1004 |         'c1',
1005 |         'd1',
1006 |         'e1',
1007 |         'f1',
1008 |         'g1',
1009 |         'h1',
1010 |     ]),
1011 |     (exports.WHITE = 'w'),
1012 |     (exports.validateFen = validateFen),
1013 |     (exports.xoroshiro128 = xoroshiro128),
1014 |     Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' }),
1015 |     exports
1016 | );
1017 | })({});
1018 |
1019 | globalThis.TSChessShared = TSChessShared;
1020 | </script>
1021 |
1022 | <script id="ai-worker-source-1" type="text/plain">
1023 |     !(function () {
1024 |         'use strict';
1025 |         const sharedCore = globalThis.TSChessShared;
1026 |         if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
1027 |         const Chess = sharedCore.Chess;
1028 |         sharedCore.SQUARES;
1029 |         const HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
1030 |             PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 },
1031 |             STARTING_NON_PAWN_MATERIAL =
1032 |                 2 * (PIECE_VALUES.q + 2 * PIECE_VALUES.r + 2 * PIECE_VALUES.b + 2 * PIECE_VALUES.n),
1033 |             OPENING_PHASE_END_MATERIAL = STARTING_NON_PAWN_MATERIAL / 2;
1034 |         function searchRoot(chess, depth, ctx, previousBestMove) {
1035 |             const candidates = [],
1036 |                 legalMoves = orderMoves(chess.moves({ verbose: !0 })),
1037 |                 for (const move of legalMoves) {
1038 |                     (checkTime(ctx), chess.move(toMoveCommand(move)));
1039 |                     const score = -negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
1040 |                     (chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score }));
1041 |                 }
1042 |             if (0 === candidates.length) throw new SearchTimeout();
1043 |             const principal = (function selectBestRootMove(candidates) {
1044 |                 return candidates.reduce((currentBest, candidate) =>
1045 |                     candidate.score > currentBest.score ? candidate : currentBest,
1046 |                     {});
1047 |             })(candidates),
1048 |                 selection = (function selectRootMove(candidates, settings, best) {
1049 |                     const level = settings.randomness / 30,
1050 |                         windowCp = 20 + 150 * level,
1051 |                         floorScore = best.score - windowCp,
1052 |                         ticketPower = 4 - 1.5 * level;
1053 |                     if (
1054 |                         (function shouldForceBestMove(settings, best) {
1055 |                             return (

```

```

1056 |         0 === settings.randomness ||
1057 |         (function isMateProtectedScore(score) {
1058 |             return Math.abs(score) ≥ 99e4;
1059 |         })(best.score) ||
1060 |         ('hard' === settings.preset && best.score < 0)
1061 |     );
1062 | })(settings, best)
1063 | )
1064 | return {
1065 |     selected: best,
1066 |     debug: createDeterministicDebug(
1067 |         candidates,
1068 |         best,
1069 |         best,
1070 |         windowCp,
1071 |         floorScore,
1072 |         ticketPower,
1073 |     ),
1074 | };
1075 | const ticketCandidates = (function selectTicketCandidates(candidates, settings, best) {
1076 |     return 'hard' === settings.preset && best.score > 0
1077 |     ? candidates.filter((candidate) => candidate.score > 0)
1078 |     : candidates;
1079 | })(candidates, settings, best),
1080 | weightedCandidates = ticketCandidates
1081 |     .map((candidate) => ({
1082 |         candidate: candidate,
1083 |         tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
1084 |     })))
1085 |     .filter(({ tickets: tickets }) => tickets > 0),
1086 | totalTickets = weightedCandidates.reduce(
1087 |     (total, { tickets: tickets }) => total + tickets,
1088 |     0,
1089 | );
1090 | if (totalTickets ≤ 0)
1091 |     return {
1092 |         selected: best,
1093 |         debug: createDeterministicDebug(
1094 |             candidates,
1095 |             best,
1096 |             best,
1097 |             windowCp,
1098 |             floorScore,
1099 |             ticketPower,
1100 |         ),
1101 |     };
1102 | let draw = Math.random() * totalTickets,
1103 | selected = best;
1104 | for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
1105 |     if (((draw -= tickets), draw ≤ 0)) {
1106 |         selected = candidate;
1107 |         break;
1108 |     }
1109 | return {
1110 |     selected: selected,
1111 |     debug: createWeightedDebug(
1112 |         candidates,
1113 |         best,
1114 |         selected,
1115 |         weightedCandidates,
1116 |         windowCp,
1117 |         floorScore,
1118 |         ticketPower,
1119 |         totalTickets,
1120 |     ),
1121 | };
1122 | })(candidates, ctx.settings, principal);
1123 | return { selected: selection.selected, principal: principal, debug: selection.debug };
1124 | }
1125 | function shouldStopAfterStableDepth(result, previousPrincipal, settings) {
1126 |     return (

```



```

1127 |         'hard' === settings.preset &&
1128 |         null !== previousPrincipal &&
1129 |         !!isSameMoveCommand(previousPrincipal.move, result.principal.move) &&
1130 |         !(Math.abs(previousPrincipal.score - result.principal.score) > 90) &&
1131 |         (function rootScoreGap(debug) {
1132 |             const [bestCandidate, secondCandidate] = debug.candidates;
1133 |             return bestCandidate && secondCandidate ? bestCandidate.score - secondCandidate.score : 1e7;
1134 |         })(result.debug) ≥ 120
1135 |     );
1136 | }
1137 |
1138 | function negamax(chess, depth, alpha, beta, ply, ctx) {
1139 |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1140 |     if (depth ≤ 0)
1141 |         return ctx.settings.quiescence
1142 |             ? quiescence(chess, alpha, beta, ply, ctx)
1143 |             : evaluatePosition(chess);
1144 |     let bestScore = -1e7,
1145 |         currentAlpha = alpha;
1146 |     const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
1147 |     for (const move of legalMoves) {
1148 |         chess.move(toMoveCommand(move));
1149 |         const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
1150 |         if (
1151 |             (chess.undo(),
1152 |             (bestScore = Math.max(bestScore, score)),
1153 |             (currentAlpha = Math.max(currentAlpha, score)),
1154 |             currentAlpha ≥ beta)
1155 |         ) break;
1156 |     }
1157 |     return bestScore;
1158 | }
1159 |
1160 | function quiescence(chess, alpha, beta, ply, ctx) {
1161 |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
1162 |     let currentAlpha = alpha;
1163 |     const standPat = evaluatePosition(chess);
1164 |     if (standPat ≥ beta) return beta;
1165 |     if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
1166 |     const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
1167 |         tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
1168 |     for (const move of tacticalMoves) {
1169 |         chess.move(toMoveCommand(move));
1170 |         const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
1171 |         if ((chess.undo(), score ≥ beta)) return beta;
1172 |         currentAlpha = Math.max(currentAlpha, score);
1173 |     }
1174 |     return currentAlpha;
1175 | }
1176 |
1177 | function evaluatePosition(chess) {
1178 |     const board = chess.board(),
1179 |         pawns = [],
1180 |         bishops = { w: 0, b: 0 },
1181 |         kings = {};
1182 |     let whiteScore = 0,
1183 |         blackScore = 0,
1184 |         nonPawnMaterial = 0;
1185 |     for (const row of board)
1186 |         for (const pieceOnSquare of row) {
1187 |             if (null === pieceOnSquare) continue;
1188 |             const { color, type, square } = pieceOnSquare,
1189 |                 pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
1190 |             ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
1191 |             'p' === type
1192 |                 ? pawns.push({ color, file: fileIndex(square), rank: rankNumber(square) })
1193 |                 : ('k' !== type && (nonPawnMaterial += PIECE_VALUES[type]),
1194 |                 'b' === type && (bishops[color] += 1),
1195 |                 'k' === type && (kings[color] = square)));
1196 |         }
1197 |     ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
1198 |     (blackScore += bishops.b ≥ 2 ? 35 : 0),
1199 |     (whiteScore += pawnStructureScore(pawns, 'w'))),

```

```

1198 |         (blackScore += pawnStructureScore(pawns, 'b')),
1199 |         (whiteScore += openingPrinciplesScore(board, pawns, 'w', nonPawnMaterial)),
1200 |         (blackScore += openingPrinciplesScore(board, pawns, 'b', nonPawnMaterial)),
1201 |         (whiteScore += developmentScore(board, 'w')),
1202 |         (blackScore += developmentScore(board, 'b')),
1203 |         (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
1204 |         (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
1205 |     let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
1206 |     return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
1207 | }
1208 |
1209 | function pieceSquareScore(type, color, square) {
1210 |     const file = fileIndex(square),
1211 |           relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
1212 |           centerDistance = Math.abs(file - 3.5) + Math.abs(rankNumber(square) - 4.5);
1213 |     switch (type) {
1214 |         case 'p':
1215 |             return 7 * relativeRank + (file >= 2 && file <= 5 ? 8 : 0);
1216 |         case 'n':
1217 |             return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
1218 |         case 'b':
1219 |             return 30 - 7 * centerDistance;
1220 |         case 'r':
1221 |             return (relativeRank >= 7 ? 18 : 0) + (0 === file || 7 === file ? 4 : 0);
1222 |         case 'q':
1223 |             return 14 - 3 * centerDistance;
1224 |         case 'k':
1225 |             return relativeRank >= 7 ? -18 : 0;
1226 |     }
1227 | }
1228 |
1229 | function pawnStructureScore(pawns, color) {
1230 |     const ownPawns = pawns.filter((pawn) => pawn.color === color),
1231 |           fileCounts = Array.from({length: 8}, () => 0);
1232 |     let score = 0;
1233 |     for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
1234 |     for (const pawn of ownPawns) {
1235 |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1236 |         (fileCounts[pawn.file] > 1 && (score -= 14),
1237 |          0 === (fileCounts[pawn.file - 1] ?? 0) &&
1238 |          0 === (fileCounts[pawn.file + 1] ?? 0) &&
1239 |          (score -= 10),
1240 |          isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
1241 |     }
1242 |     return score;
1243 | }
1244 |
1245 | function isPassedPawn(pawn, pawns) {
1246 |     const enemyColor = (function oppositeColor(color) {
1247 |         return 'w' === color ? 'b' : 'w';
1248 |     })(pawn.color);
1249 |     return !pawns.some(
1250 |         (candidate) =>
1251 |             !(candidate.color === enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
1252 |             ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
1253 |     );
1254 | }
1255 |
1256 | function openingPhaseWeight(nonPawnMaterial) {
1257 |     return clamp(
1258 |         (nonPawnMaterial - OPENING_PHASE_END_MATERIAL) /
1259 |         (STARTING_NON_PAWN_MATERIAL - OPENING_PHASE_END_MATERIAL),
1260 |         0,
1261 |         1,
1262 |     );
1263 | }
1264 |
1265 | function openingPrinciplesScore(board, pawns, color, nonPawnMaterial) {
1266 |     const phase = openingPhaseWeight(nonPawnMaterial);
1267 |     if (phase <= 0) return 0;
1268 |     let score = (function centralLineOpeningScore(board, color) {
1269 |         let score = 0;
1270 |         return (
1271 |             hasPawnOnSquare(board, color, 'w' === color ? 'd2' : 'd7') || (score += 10),
1272 |             hasPawnOnSquare(board, color, 'w' === color ? 'e2' : 'e7') || (score += 12),
1273 |             score
1274 |         );
1275 |     })(board, color);
1276 |     return (score * phase + 100 * (1 - phase));
1277 | }

```

```

1269 |         .);
1270 |     })(board, color),
1271 |     developedCenterPawns = 0;
1272 |     for (const pawn of pawns) {
1273 |         if (pawn.color !== color) continue;
1274 |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1275 |         3 === pawn.file || 4 === pawn.file
1276 |         ? (4 === relativeRank || 5 === relativeRank
1277 |           ? (score += 76)
1278 |             : 3 === relativeRank && (score += 30),
1279 |           relativeRank > 3 && (developedCenterPawns += 1))
1280 |         : (2 !== pawn.file && 5 !== pawn.file) ||
1281 |           (4 === relativeRank || 5 === relativeRank
1282 |           ? (score += 18)
1283 |             : 3 === relativeRank && (score += 6));
1284 |     }
1285 |     return (developedCenterPawns >= 2 && (score += 16), Math.round(score * phase));
1286 | }
1287 | function hasPawnOnSquare(board, color, square) {
1288 |     for (const row of board)
1289 |         for (const piece of row)
1290 |             if (piece?.color === color && 'p' === piece.type && piece.square === square) return !0;
1291 |     return !1;
1292 | }
1293 | function developmentScore(board, color) {
1294 |     const homeRank = 'w' === color ? '1' : '8',
1295 |           knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
1296 |           bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
1297 |     let score = 0;
1298 |     for (const row of board)
1299 |         for (const piece of row)
1300 |             null !== piece &&
1301 |             piece.color === color &&
1302 |             ('n' !== piece.type || knightHomeSquares.has(piece.square)) || (score += 18),
1303 |             ('b' !== piece.type || bishopHomeSquares.has(piece.square)) || (score += 16),
1304 |             'q' === piece.type &&
1305 |             piece.square.endsWith(homeRank) &&
1306 |             hasMovedMinorPieces(board, color) &&
1307 |             (score -= 12));
1308 |     return score;
1309 | }
1310 | function hasMovedMinorPieces(board, color) {
1311 |     const homeSquares =
1312 |       'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
1313 |     for (const row of board)
1314 |         for (const piece of row)
1315 |             if (
1316 |               null !== piece &&
1317 |               piece.color === color &&
1318 |               ('n' === piece.type || 'b' === piece.type) &&
1319 |               homeSquares.has(piece.square)
1320 |             )
1321 |                 return !1;
1322 |     return !0;
1323 | }
1324 | function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
1325 |     if (!kingSquare || nonPawnMaterial < 1800) return 0;
1326 |     const kingFile = fileIndex(kingSquare),
1327 |           kingRank = rankNumber(kingSquare),
1328 |           shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
1329 |     let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
1330 |     for (let file = kingFile - 1; file <= kingFile + 1; file += 1)
1331 |         file < 0 ||
1332 |         file > 8 ||
1333 |         (pawns.some(
1334 |           (pawn) => pawn.color === color && pawn.file === file && pawn.rank === shieldRank,
1335 |           )
1336 |         ? (score += 14)
1337 |           : (score +=
1338 |             openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) ?? -10));
1339 |     return score;

```

```

1340 | . . . . }
1341 | . . . . function openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) {
1342 | . . . .   const homeKingSquare = 'w' === color ? 'e1' : 'e8',
1343 | . . . .   phase = openingPhaseWeight(nonPawnMaterial);
1344 | . . . .   return kingSquare === homeKingSquare || phase ≤ 0 || (3 !== file && 4 !== file)
1345 | . . . .     ? null
1346 | . . . .     : pawns.some((pawn) => {
1347 | . . . .       if (pawn.color !== color || pawn.file !== file) return !1;
1348 | . . . .       const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
1349 | . . . .       return 3 === relativeRank || 4 === relativeRank;
1350 | . . . .     })
1351 | . . . .     ? Math.round(24 * phase - 10)
1352 | . . . .     : null;
1353 | . . . . }
1354 | . . . . function terminalScore(chess, ply) {
1355 | . . . .   return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
1356 | . . . . }
1357 | . . . . function orderMoves(moves, preferredMove) {
1358 | . . . .   return [... moves].sort(
1359 | . . . .     (left, right) =>
1360 | . . . .     moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),
1361 | . . . .   );
1362 | . . . . }
1363 | . . . . function moveOrderingScore(move, preferredMove) {
1364 | . . . .   let score = 0;
1365 | . . . .   return (
1366 | . . . .     preferredMove &&
1367 | . . . .     (function isSameMove(move, command) {
1368 | . . . .       return (
1369 | . . . .         move.from === command.from &&
1370 | . . . .         move.to === command.to &&
1371 | . . . .         (move.promotion ?? void 0) === (command.promotion ?? void 0)
1372 | . . . .       );
1373 | . . . .     })(move, preferredMove) &&
1374 | . . . .     (score += 2e6),
1375 | . . . .     move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
1376 | . . . .     move.isCapture() &&
1377 | . . . .     (score +=
1378 | . . . .       12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
1379 | . . . .       PIECE_VALUES[move.piece]),
1380 | . . . .     move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] * 8e3),
1381 | . . . .     (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
1382 | . . . .     (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
1383 | . . . .     score
1384 | . . . .   );
1385 | . . . . }
1386 | . . . . function checkTime(ctx) {
1387 | . . . .   if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
1388 | . . . .   const now = Date.now();
1389 | . . . .   if (
1390 | . . . .     (now - ctx.lastProgressAt ≥ 100 &&
1391 | . . . .     publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
1392 | . . . .     now ≥ ctx.deadline)
1393 | . . . .   )
1394 | . . . .     throw new SearchTimeout();
1395 | . . . . }
1396 | . . . . function publishSearchProgress(
1397 | . . . .   ctx,
1398 | . . . .   progressState,
1399 | . . . .   elapsedMs = Date.now() - ctx.startTime,
1400 | . . . .   progressAt = Date.now(),
1401 | . . . . ) {
1402 | . . . .   (ctx.progressState = progressState),
1403 | . . . .   (ctx.lastProgressAt = progressAt),
1404 | . . . .   ctx.onProgress?.({
1405 | . . . .     kind: 'progress',
1406 | . . . .     id: ctx.requestId,
1407 | . . . .     move: progressState.move,
1408 | . . . .     depthReached: progressState.depthReached,
1409 | . . . .     nodes: ctx.nodes,
1410 | . . . .     elapsedMs: elapsedMs,

```

```

1411 |         score: progressState.score,
1412 |         debug: progressState.debug,
1413 |     }));
1414 |     }
1415 |     function isTacticalMove(move) {
1416 |         return (
1417 |             move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
1418 |         );
1419 |     }
1420 |     function createDeterministicDebug(
1421 |         candidates,
1422 |         principal,
1423 |         selected,
1424 |         windowCp,
1425 |         floorScore,
1426 |         ticketPower,
1427 |     ) {
1428 |         return {
1429 |             candidates: sortCandidatesByScore(candidates).map((candidate) => ({
1430 |                 move: candidate.move,
1431 |                 san: candidate.san,
1432 |                 score: candidate.score,
1433 |                 tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1434 |                 probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
1435 |                 selected: isSameMoveCommand(candidate.move, selected.move),
1436 |                 principal: isSameMoveCommand(candidate.move, principal.move),
1437 |             })),
1438 |             bestScore: principal.score,
1439 |             windowCp: windowCp,
1440 |             floorScore: floorScore,
1441 |             ticketPower: ticketPower,
1442 |             totalTickets: 1,
1443 |         };
1444 |     }
1445 |     function createWeightedDebug(
1446 |         candidates,
1447 |         principal,
1448 |         selected,
1449 |         weightedCandidates,
1450 |         windowCp,
1451 |         floorScore,
1452 |         ticketPower,
1453 |         totalTickets,
1454 |     ) {
1455 |         return {
1456 |             candidates: sortCandidatesByScore(candidates).map((candidate) => {
1457 |                 const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
1458 |                     isSameMoveCommand(weighted.move, candidate.move),
1459 |                 ),
1460 |                 tickets = weightedCandidate ? tickets ?? 0;
1461 |                 return {
1462 |                     move: candidate.move,
1463 |                     san: candidate.san,
1464 |                     score: candidate.score,
1465 |                     tickets: tickets,
1466 |                     probability: totalTickets > 0 ? tickets / totalTickets : 0,
1467 |                     selected: isSameMoveCommand(candidate.move, selected.move),
1468 |                     principal: isSameMoveCommand(candidate.move, principal.move),
1469 |                 };
1470 |             }),
1471 |             bestScore: principal.score,
1472 |             windowCp: windowCp,
1473 |             floorScore: floorScore,
1474 |             ticketPower: ticketPower,
1475 |             totalTickets: totalTickets,
1476 |         };
1477 |     }
1478 |     function sortCandidatesByScore(candidates) {
1479 |         return [...candidates].sort((left, right) => right.score - left.score);
1480 |     }
1481 |     function toMoveCommand(move) {

```

```

1482 |         return move.promotion
1483 |       ? { from: move.from, to: move.to, promotion: move.promotion }
1484 |       : { from: move.from, to: move.to };
1485 |     }
1486 |     function isSameMoveCommand(left, right) {
1487 |       return (
1488 |         left.from === right.from &&
1489 |         left.to === right.to &&
1490 |         (left.promotion ?? void 0) === (right.promotion ?? void 0)
1491 |       );
1492 |     }
1493 |     function fileIndex(square) {
1494 |       return square.charCodeAt(0) - 97;
1495 |     }
1496 |     function rankNumber(square) {
1497 |       return Number(square[1]);
1498 |     }
1499 |     function clamp(value, min, max) {
1500 |       return Math.min(max, Math.max(min, value));
1501 |     }
1502 |     class SearchTimeout extends Error {
1503 |       const workerScope = globalThis;
1504 |       workerScope.addEventListener('message', (event) => {
1505 |         const response = (function searchBestMove(request, onProgress) {
1506 |           const chess = new Chess(request.fen),
1507 |                 settings = (function normalizeAiSettings(settings) {
1508 |                     const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
1509 |                     return {
1510 |                       preset: settings.preset,
1511 |                       maxDepth:
1512 |                         ((depth = settings.maxDepth),
1513 |                          (preset = settings.preset),
1514 |                          'hard' === preset ? HARD_MAX_DEPTH : clamp(Math.round(depth), 1, 5)),
1515 |                       timeLimitMs: timeLimitMs,
1516 |                       randomness: 30,
1517 |                       quiescence: !0,
1518 |                     };
1519 |                     var depth, preset;
1520 |                 })(request.settings),
1521 |                 legalMoves = chess.moves({ verbose: !0 }),
1522 |                 startTime = Date.now(),
1523 |                 ctx = {
1524 |                   requestId: request.id,
1525 |                   settings: settings,
1526 |                   deadline: startTime + settings.timeLimitMs,
1527 |                   startTime: startTime,
1528 |                   nodes: 0,
1529 |                   onProgress: onProgress,
1530 |                   lastProgressAt: startTime,
1531 |                   progressState: { move: null, depthReached: 0, score: 0 },
1532 |                 };
1533 |                 if (0 === legalMoves.length)
1534 |                   return {
1535 |                     id: request.id,
1536 |                     move: null,
1537 |                     depthReached: 0,
1538 |                     nodes: 0,
1539 |                     elapsedMs: 0,
1540 |                     score: terminalScore(chess, 0),
1541 |                   };
1542 |                 let debug,
1543 |                     best = null,
1544 |                     previousBestMove = null,
1545 |                     previousPrincipal = null,
1546 |                     depthReached = 0;
1547 |                 for (let depth = 1; depth <= settings.maxDepth; depth += 1) {
1548 |                   try {
1549 |                     const result = searchRoot(chess, depth, ctx, previousBestMove),
1550 |                           elapsedMs = Date.now() - startTime;
1551 |                     if (
1552 |                       ((best = result.selected),

```

```

1553 |         (debug = result.debug),
1554 |         (previousBestMove = result.principal.move),
1555 |         (depthReached = depth),
1556 |         publishSearchProgress(
1557 |             ctx,
1558 |             { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
1559 |             elapsedMs,
1560 |         ),
1561 |         shouldStopAfterStableDepth(result, previousPrincipal, settings))
1562 |     )
1563 |     break;
1564 |     previousPrincipal = result.principal;
1565 | } catch (error) {
1566 |     if (error instanceof SearchTimeout) break;
1567 |     throw error;
1568 | }
1569 | if (Date.now() >= ctx.deadline) break;
1570 | }
1571 | return (
1572 |     (best ??= (function chooseFallbackMove(chess) {
1573 |         const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
1574 |         if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
1575 |         chess.move(toMoveCommand(move));
1576 |         const score = -evaluatePosition(chess);
1577 |         return (chess.undo(), { move: toMoveCommand(move), score: score });
1578 |     })(chess)),
1579 |     {
1580 |         id: request.id,
1581 |         move: best.move,
1582 |         depthReached: depthReached,
1583 |         nodes: ctx.nodes,
1584 |         elapsedMs: Date.now() - startTime,
1585 |         score: best.score,
1586 |         debug: debug,
1587 |     }
1588 | );
1589 | })(event.data, (progress) => {
1590 |     workerScope.postMessage(progress);
1591 | });
1592 | workerScope.postMessage({ ... response, kind: 'result' });
1593 | });
1594 | })();
1595 | </script>
1596 |
1597 | <script>
1598 |     (() => {
1599 |         const element = document.getElementById('ts-chess-shared-core-source');
1600 |         if (!element) {
1601 |             throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
1602 |         }
1603 |         new Function(element.textContent || '')();
1604 |     })();
1605 | </script>
1606 | <script src="/main.js" defer></script>
1607 | <link rel="stylesheet" href="/style.css" />
1608 | </head>
1609 | <body>
1610 |     <main id="chess-app" class="chess-app">
1611 |         <aside class="control-panel" aria-label="게임 컨트롤">
1612 |             <section class="control-group" aria-label="모드">
1613 |                 <span class="control-label">모드</span>
1614 |                 <div class="segmented-control" role="group" aria-label="플레이 모드">
1615 |                     <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
1616 |                     <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
1617 |                 </div>
1618 |             </section>
1619 |
1620 |             <section id="color-controls" class="control-group" aria-label="진영">
1621 |                 <span class="control-label">진영</span>
1622 |                 <div class="segmented-control" role="group" aria-label="인간 진영">
1623 |                     <button class="segmented-button" type="button" data-color="w">백</button>

```

```

1624 |     <button class="segmented-button" type="button" data-color="b">흑</button>
1625 |   </div>
1626 | </section>
1627 |
1628 |   <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
1629 |     <span class="control-label">AI 설정</span>
1630 |     <div
1631 |       class="segmented-control segmented-control-three"
1632 |       role="group"
1633 |       aria-label="AI 프리셋"
1634 |     >
1635 |       <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
1636 |       <button class="segmented-button" type="button" data-preset="normal">보통</button>
1637 |       <button class="segmented-button" type="button" data-preset="hard">어려움</button>
1638 |     </div>
1639 |     <label class="range-control">
1640 |       <span>최대 깊이<strong id="ai-depth-value"></strong></span>
1641 |       <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
1642 |     </label>
1643 |     <label class="range-control">
1644 |       <span>시간 제한<strong id="ai-time-value"></strong></span>
1645 |       <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
1646 |     </label>
1647 |   </section>
1648 |
1649 |   <section class="control-group clock-settings" aria-label="시간 설정">
1650 |     <span class="control-label">시간</span>
1651 |     <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
1652 |       <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
1653 |       <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
1654 |       <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
1655 |       <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
1656 |       <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
1657 |     </div>
1658 |   </section>
1659 |
1660 |   <section class="action-row" aria-label="게임 동작">
1661 |     <button id="new-game-button" class="primary-button" type="button">새 게임</button>
1662 |     <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
1663 |   </section>
1664 | </aside>
1665 |
1666 |   <section class="board-stage" aria-label="체스판">
1667 |     <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
1668 |   </section>
1669 |
1670 |   <aside class="clock-panel" aria-label="체스 타이머">
1671 |     <div id="clock-display" class="clock-display">
1672 |       <div id="white-clock" class="clock-card">
1673 |         <span class="clock-side">백</span>
1674 |         <strong id="white-clock-time" class="clock-time"></strong>
1675 |       </div>
1676 |       <div id="black-clock" class="clock-card">
1677 |         <span class="clock-side">흑</span>
1678 |         <strong id="black-clock-time" class="clock-time"></strong>
1679 |       </div>
1680 |     </div>
1681 |   </aside>
1682 |
1683 |   <aside class="info-panel" aria-label="게임 정보">
1684 |     <section class="status-panel" aria-live="polite">
1685 |       <span class="control-label">상태</span>
1686 |       <strong id="game-status" class="game-status"></strong>
1687 |       <dl class="status-grid">
1688 |         <div>
1689 |           <dt>차례</dt>
1690 |           <dd id="turn-meta"></dd>
1691 |         </div>
1692 |         <div>
1693 |           <dt>모드</dt>
1694 |           <dd id="mode-meta"></dd>

```



```

1695 | | | | |</div>
1696 | | | | |<div>
1697 | | | | |<dt>수순</dt>
1698 | | | | |<dd id="move-meta"></dd>
1699 | | | | |</div>
1700 | | | | |</dl>
1701 | | | | |</section>
1702 | | | | |
1703 | | | | |<section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI·정보" hidden>
1704 | | | | |<div class="ai-candidate-header">
1705 | | | | |<span class="control-label">AI·정보</span>
1706 | | | | |<span id="ai-candidate-meta" class="ai-candidate-meta"></span>
1707 | | | | |</div>
1708 | | | | |<dl class="ai-info-grid">
1709 | | | | |<div>
1710 | | | | |<dt>상태</dt>
1711 | | | | |<dd id="ai-info-status"></dd>
1712 | | | | |</div>
1713 | | | | |<div>
1714 | | | | |<dt>난이도</dt>
1715 | | | | |<dd id="ai-info-preset"></dd>
1716 | | | | |</div>
1717 | | | | |<div>
1718 | | | | |<dt>인간</dt>
1719 | | | | |<dd id="ai-info-human"></dd>
1720 | | | | |</div>
1721 | | | | |<div>
1722 | | | | |<dt>depth</dt>
1723 | | | | |<dd id="ai-info-depth"></dd>
1724 | | | | |</div>
1725 | | | | |<div>
1726 | | | | |<dt>nodes</dt>
1727 | | | | |<dd id="ai-info-nodes"></dd>
1728 | | | | |</div>
1729 | | | | |<div>
1730 | | | | |<dt>score</dt>
1731 | | | | |<dd id="ai-info-score"></dd>
1732 | | | | |</div>
1733 | | | | |</dl>
1734 | | | | |<div class="ai-candidate-table" role="table" aria-label="AI·후보·수·평가">
1735 | | | | |<div class="ai-candidate-row ai-candidate-head" role="row">
1736 | | | | |<span role="columnheader">SAN</span>
1737 | | | | |<span role="columnheader">cp</span>
1738 | | | | |<span role="columnheader">weight</span>
1739 | | | | |<span role="columnheader">prob</span>
1740 | | | | |</div>
1741 | | | | |<div id="ai-candidate-list" class="ai-candidate-list"></div>
1742 | | | | |</div>
1743 | | | | |</section>
1744 | | | | |
1745 | | | | |<section class="history-panel" aria-label="기보">
1746 | | | | |<div class="history-header">
1747 | | | | |<span class="control-label">기보</span>
1748 | | | | |</div>
1749 | | | | |<div id="move-list" class="move-list"></div>
1750 | | | | |</section>
1751 | | | | |</aside>
1752 | | | | |
1753 | | | | |<div id="promotion-overlay" class="promotion-overlay" hidden>
1754 | | | | |<div
1755 | | | | |class="promotion-dialog"
1756 | | | | |role="dialog"
1757 | | | | |aria-modal="true"
1758 | | | | |aria-labelledby="promotion-title"
1759 | | | | |>
1760 | | | | |<h2 id="promotion-title">승격</h2>
1761 | | | | |<div
1762 | | | | |id="promotion-choices"
1763 | | | | |class="promotion-choices"
1764 | | | | |role="group"
1765 | | | | |aria-label="승격·기물"

```

```

1766 | |></div>
1767 | |</div>
1768 | |</div>
1769 | |</main>
1770 | |</body>
1771 | </html>
1772 |

```

==== main.js (33823 B / 04a8e344941e) ====

```

1 | const sharedCore = globalThis.TSChessShared;
2 | if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
3 | const Chess = sharedCore.Chess,
4 |   SQUARES = sharedCore.SQUARES,
5 |   HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
6 |   AI_PRESETS = {
7 |     easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
8 |     normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
9 |     hard: {
10 |       preset: 'hard',
11 |       maxDepth: HARD_MAX_DEPTH,
12 |       timeLimitMs: 5e3,
13 |       randomness: 30,
14 |       quiescence: !0,
15 |     },
16 |   },
17 |   DEFAULT_AI_SETTINGS = { ...AI_PRESETS.normal };
18 | function normalizeAiSettings(settings) {
19 |   const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
20 |   return {
21 |     preset: settings.preset,
22 |     maxDepth:
23 |       ((depth = settings.maxDepth),
24 |       (preset = settings.preset),
25 |       'hard' === preset
26 |       ? HARD_MAX_DEPTH
27 |       : (function clamp(value, min, max) {
28 |         return Math.min(max, Math.max(min, value));
29 |       })(Math.round(depth), 1, 5)),
30 |     timeLimitMs: timeLimitMs,
31 |     randomness: 30,
32 |     quiescence: !0,
33 |   };
34 |   var depth, preset;
35 | }
36 | const jsContent = (() => {
37 |   const sharedElement = document.getElementById('ts-chess-shared-core-source');
38 |   const workerElement = document.getElementById('ai-worker-source-1');
39 |   if (!sharedElement) {
40 |     throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
41 |   }
42 |   if (!workerElement) {
43 |     throw new Error('Missing inline worker source: ai-worker-source-1');
44 |   }
45 |   return (sharedElement.textContent || '') + '\n' + (workerElement.textContent || '');
46 | })(),
47 | blob =
48 |   'undefined' !== typeof self &&
49 |   self.Blob &&
50 |   new Blob([(self.URL || self.webkitURL).revokeObjectURL(self.location.href);', jsContent], {
51 |     type: 'text/javascript;charset=utf-8',
52 |   });
53 | function WorkerWrapper(options) {
54 |   let objURL;
55 |   try {
56 |     if (((objURL = blob && (self.URL || self.webkitURL).createObjectURL(blob)), !objURL)) throw '';
57 |     const worker = new Worker(objURL, { name: options?.name });
58 |     return (
59 |       worker.addEventListener('error', () => {
60 |         (self.URL || self.webkitURL).revokeObjectURL(objURL);
61 |       }),
62 |       worker

```

```

63 |     });
64 |   }.catch(e) {
65 |     return new Worker('data:text/javascript;charset=utf-8,' + encodeURIComponent(jsContent), {
66 |       name: options?.name,
67 |     });
68 |   }
69 | }
70 | const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
71 |   BLACK_FILES = [...BOARD_FILES].reverse(),
72 |   WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
73 |   BLACK_RANKS = [...WHITE_RANKS].reverse(),
74 |   PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
75 |   SQUARE_SET = new Set(SQUARES),
76 |   AI_WHITE_OPENING_PRESET = [
77 |     { from: 'e2', to: 'e4', weight: 28 },
78 |     { from: 'd2', to: 'd4', weight: 25 },
79 |     { from: 'g1', to: 'f3', weight: 24 },
80 |     { from: 'c2', to: 'c4', weight: 20 },
81 |     { from: 'g2', to: 'g3', weight: 9 },
82 |     { from: 'b2', to: 'b3', weight: 6 },
83 |   ],
84 |   CLOCK_PRESETS = {
85 |     '1/1': { baseMs: 6e4, incrementMs: 1e3 },
86 |     '3/2': { baseMs: 18e4, incrementMs: 2e3 },
87 |     '5/3': { baseMs: 3e5, incrementMs: 3e3 },
88 |     '10/5': { baseMs: 6e5, incrementMs: 5e3 },
89 |     '15/10': { baseMs: 9e5, incrementMs: 1e4 },
90 |   },
91 |   PIECE_GLYPHS = {
92 |     w: { p: '♔', n: '♖', b: '♜', r: '♞', q: '♙', k: '♚' },
93 |     b: { p: '♚', n: '♛', b: '♞', r: '♝', q: '♜', k: '♗' },
94 |   };
95 | function requireElement(selector, root) {
96 |   const element = root.querySelector(selector);
97 |   if (!element) throw new Error(`Missing element: ${selector}`);
98 |   return element;
99 | }
100 | function createCoordinate(type, label) {
101 |   const coordinate = document.createElement('span');
102 |   return (
103 |     (coordinate.className = `coordinate coordinate-${type}`),
104 |     (coordinate.textContent = label),
105 |     coordinate
106 |   );
107 | }
108 | function toSquare(file, rank) {
109 |   return `${file}${rank}`;
110 | }
111 | function createClockState(preset) {
112 |   const baseMs = CLOCK_PRESETS[preset].baseMs;
113 |   return { w: baseMs, b: baseMs };
114 | }
115 | function colorName(color) {
116 |   return 'w' === color ? '백' : '흑';
117 | }
118 | function presetName(preset) {
119 |   switch (preset) {
120 |     case 'easy':
121 |       return '쉬움';
122 |     case 'normal':
123 |       return '보통';
124 |     case 'hard':
125 |       return '어려움';
126 |   }
127 | }
128 | function formatSeconds(milliseconds) {
129 |   return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
130 | }
131 | function formatDebugScore(score) {
132 |   return Math.abs(score) ≥ 9e5
133 |     ? `score > 0`

```

```

134 |     .? : '+mate'
135 |     .: : '-mate'
136 |     .: : score > 0
137 |     .? : `+${Math.round(score)}`
138 |     .: : String(Math.round(score));
139 | }
140 | function oppositeColor(color) {
141 |     return 'w' === color ? 'b' : 'w';
142 | }
143 | function getSquareLabel(square, piece) {
144 |     return piece ? `${square}-${colorName(piece.color)}-${pieceName(piece.type)}` : ` ${square} 빈 칸`;
145 | }
146 | function pieceName(piece) {
147 |     switch (piece) {
148 |     case 'p':
149 |         return '폰';
150 |     case 'n':
151 |         return '나이트';
152 |     case 'b':
153 |         return '비숍';
154 |     case 'r':
155 |         return '룩';
156 |     case 'q':
157 |         return '퀸';
158 |     case 'k':
159 |         return '킹';
160 |     }
161 | }
162 | !(function createChessApp() {
163 |     const root = requireElement('#chess-app', document),
164 |           boardElement = requireElement('#chess-board', root),
165 |           colorControlsElement = requireElement('#color-controls', root),
166 |           aiControlsElement = requireElement('#ai-controls', root),
167 |           statusElement = requireElement('#game-status', root),
168 |           turnMetaElement = requireElement('#turn-meta', root),
169 |           modeMetaElement = requireElement('#mode-meta', root),
170 |           moveMetaElement = requireElement('#move-meta', root),
171 |           aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
172 |           aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
173 |           aiInfoStatusElement = requireElement('#ai-info-status', root),
174 |           aiInfoPresetElement = requireElement('#ai-info-preset', root),
175 |           aiInfoHumanElement = requireElement('#ai-info-human', root),
176 |           aiInfoDepthElement = requireElement('#ai-info-depth', root),
177 |           aiInfoNodesElement = requireElement('#ai-info-nodes', root),
178 |           aiInfoScoreElement = requireElement('#ai-info-score', root),
179 |           aiCandidateListElement = requireElement('#ai-candidate-list', root),
180 |           moveListElement = requireElement('#move-list', root),
181 |           clockDisplayElement = requireElement('#clock-display', root),
182 |           whiteClockElement = requireElement('#white-clock', root),
183 |           blackClockElement = requireElement('#black-clock', root),
184 |           whiteClockTimeElement = requireElement('#white-clock-time', root),
185 |           blackClockTimeElement = requireElement('#black-clock-time', root),
186 |           newGameButton = requireElement('#new-game-button', root),
187 |           undoButton = requireElement('#undo-button', root),
188 |           aiDepthInput = requireElement('#ai-depth-input', root),
189 |           aiDepthValue = requireElement('#ai-depth-value', root),
190 |           aiTimeInput = requireElement('#ai-time-input', root),
191 |           aiTimeValue = requireElement('#ai-time-value', root),
192 |           promotionOverlay = requireElement('#promotion-overlay', root),
193 |           promotionChoices = requireElement('#promotion-choices', root),
194 |           modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
195 |           colorButtons = Array.from(root.querySelectorAll('[data-color]')),
196 |           presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
197 |           clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
198 |           game = new Chess(),
199 |           let mode = 'local',
200 |           clockPreset = '10/5',
201 |           clockMs = createClockState(clockPreset),
202 |           activeClockColor = null,
203 |           lastClockTick = Date.now(),
204 |           clockTimerId = null,

```

```

205 |     .timedOutColor = null,
206 |     .humanColor = 'w',
207 |     .selectedSquare = null,
208 |     .legalMoves = [],
209 |     .pendingPromotion = null,
210 |     .aiThinking = !1,
211 |     .aiTimerId = null,
212 |     .aiWorker = null,
213 |     .aiRequestId = 0,
214 |     .activeAiRequestId = null,
215 |     .activeAiFen = null,
216 |     .aiSettings = { ... DEFAULT_AI_SETTINGS },
217 |     .aiProgress = null;
218 |     function selectSquare(square) {
219 |       ((selectedSquare = square),
220 |       (legalMoves = game.moves({ verbose: !0, square: square })),
221 |       render());
222 |     }
223 |     function playMove(moveCommand) {
224 |       if (!commitMove(moveCommand)) return (clearSelection(), void render());
225 |       ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
226 |     }
227 |     function resetGame() {
228 |       (cancelAiMove(),
229 |       game.reset(),
230 |       (pendingPromotion = null),
231 |       clearSelection(),
232 |       resetClockState(),
233 |       restartClockForCurrentTurn(),
234 |       render(),
235 |       scheduleAiIfNeeded());
236 |     }
237 |     function scheduleAiIfNeeded() {
238 |       'ai' !== mode ||
239 |       game.isGameOver() ||
240 |       null !== timedOutColor ||
241 |       game.turn() === humanColor ||
242 |       aiThinking ||
243 |       pendingPromotion ||
244 |       (function tryCommitAiWhiteOpeningPreset() {
245 |         if ('b' !== humanColor || 'w' !== game.turn() || game.history().length > 0) return !1;
246 |         const selectedMove = (function pickWeightedOpeningMove(moves) {
247 |           const totalWeight = moves.reduce((total, move) => total + move.weight, 0);
248 |           if (totalWeight <= 0) return null;
249 |           let draw = Math.random() * totalWeight;
250 |           for (const move of moves)
251 |             if (((draw -= move.weight), draw <= 0))
252 |               return { from: move.from, to: move.to, promotion: move.promotion };
253 |           const fallback = moves.at(-1);
254 |           return fallback
255 |             ? { from: fallback.from, to: fallback.to, promotion: fallback.promotion }
256 |             : null;
257 |         })(
258 |           AI_WHITE_OPENING_PRESET.filter((presetMove) =>
259 |             game.moves({ verbose: !0, square: presetMove.from }).some((move) =>
260 |               (function isSameMoveCommand(move, command) {
261 |                 return (
262 |                   move.from === command.from &&
263 |                   move.to === command.to &&
264 |                   (move.promotion ?? void 0) === (command.promotion ?? void 0)
265 |                 );
266 |               })(move, presetMove),
267 |             ),
268 |           ),
269 |         );
270 |         return ! (
271 |           !selectedMove ||
272 |           !commitMove(selectedMove) ||
273 |           ((aiProgress = null), clearSelection(), render(), 0)
274 |         );
275 |       })() ||

```

```

276 |         ((aiThinking -= !0),
277 |         (aiProgress = null),
278 |         render(),
279 |         (aiTimerId = window.setTimeout(() => {
280 |             ((aiTimerId = null),
281 |             (function startAiSearch() {
282 |                 aiWorker?.terminate();
283 |                 const requestId = aiRequestId + 1,
284 |                 fen = game.fen();
285 |                 ((aiRequestId = requestId),
286 |                 (activeAiRequestId = requestId),
287 |                 (activeAiFen = fen),
288 |                 (aiWorker = new WorkerWrapper()),
289 |                 aiWorker.addEventListener('message', (event) => {
290 |                     !(function handleAiMessage(message) {
291 |                         if (
292 |                             message.id === activeAiRequestId &&
293 |                             null !== activeAiFen &&
294 |                             game.fen() === activeAiFen
295 |                         ) {
296 |                             if (((aiProgress = message), 'progress' === message.kind))
297 |                                 return (renderStatus(), void renderAiCandidateDebug());
298 |                             ((aiThinking = !1),
299 |                             (activeAiRequestId = null),
300 |                             (activeAiFen = null),
301 |                             aiWorker?.terminate(),
302 |                             (aiWorker = null),
303 |                             message.move && commitMove(message.move),
304 |                             clearSelection(),
305 |                             render());
306 |                         }
307 |                     })(event.data);
308 |                 })),
309 |                 aiWorker.addEventListener('error', () => {
310 |                     activeAiRequestId === requestId &&
311 |                     ((aiThinking -= !1),
312 |                     (activeAiRequestId = null),
313 |                     (activeAiFen = null),
314 |                     aiWorker?.terminate(),
315 |                     (aiWorker = null),
316 |                     render());
317 |                 })),
318 |                 aiWorker.postMessage({
319 |                     id: requestId,
320 |                     fen: fen,
321 |                     settings: resolveAiSettingsForSearch(),
322 |                 }));
323 |             })());
324 |         }, 180));
325 |     }
326 |     function cancelAiMove() {
327 |         ((aiRequestId += 1),
328 |         (activeAiRequestId = null),
329 |         (activeAiFen = null),
330 |         (aiProgress = null),
331 |         null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
332 |         aiWorker && (aiWorker.terminate(), (aiWorker = null)),
333 |         (aiThinking = !1));
334 |     }
335 |     function commitMove(moveCommand) {
336 |         const movingColor = game.turn();
337 |         if (!flushClockElapsed()) return !1;
338 |         try {
339 |             game.move(moveCommand);
340 |         } catch {
341 |             return !1;
342 |         }
343 |         return (
344 |             (clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
345 |             restartClockForCurrentTurn(),
346 |             !0

```

```

347 |     .);
348 | }
349 | function updateAiSettings(settings) {
350 |     ((aiSettings := normalizeAiSettings(settings)),
351 |     aiThinking && cancelAiMove(),
352 |     render(),
353 |     scheduleAiIfNeeded());
354 | }
355 | function resolveAiSettingsForSearch() {
356 |     const settings := normalizeAiSettings(aiSettings);
357 |     if ('hard' ≠ settings.preset) return settings;
358 |     const aiColor := game.turn(),
359 |     remainingMs := clockMs[aiColor],
360 |     targetMs := remainingMs / 30 + 0.8 * CLOCK_PRESETS[clockPreset].incrementMs,
361 |     timeLimitMs := remainingMs ≤ 3e3 ? Math.min(targetMs, 0.35 * remainingMs) : targetMs;
362 |     return normalizeAiSettings({ ... settings, timeLimitMs: timeLimitMs });
363 | }
364 | function resetClockState() {
365 |     ((clockMs := createClockState(clockPreset)),
366 |     (timedOutColor := null),
367 |     (activeClockColor := null),
368 |     (lastClockTick := Date.now()));
369 | }
370 | function restartClockForCurrentTurn() {
371 |     if ((clearClockTimer(), game.isGameOver() || null ≠ timedOutColor))
372 |     return (activeClockColor := null), void renderClocks();
373 |     ((activeClockColor := game.turn(),
374 |     (lastClockTick := Date.now()),
375 |     (clockTimerId := window.setInterval(tickClock, 100)),
376 |     renderClocks());
377 | }
378 | function clearClockTimer() {
379 |     null ≠ clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId := null));
380 | }
381 | function tickClock() {
382 |     flushClockElapsed() && renderClocks();
383 | }
384 | function flushClockElapsed() {
385 |     if (null == activeClockColor || null ≠ timedOutColor || game.isGameOver()) return !0;
386 |     const now := Date.now(),
387 |     elapsedMs := Math.max(0, now - lastClockTick);
388 |     return (
389 |     (lastClockTick := now),
390 |     0 == elapsedMs ||
391 |     ((clockMs[activeClockColor] := Math.max(0, clockMs[activeClockColor] - elapsedMs)),
392 |     0 ≠ clockMs[activeClockColor] ||
393 |     ((function handleClockTimeout(color) {
394 |         ((timedOutColor := color),
395 |         (activeClockColor := null),
396 |         (clockMs[color] := 0),
397 |         clearClockTimer(),
398 |         cancelAiMove(),
399 |         clearSelection(),
400 |         render());
401 |     }))(activeClockColor),
402 |     !1))
403 |     .);
404 | }
405 | function clearSelection() {
406 |     ((selectedSquare := null), (legalMoves := []));
407 | }
408 | function render() {
409 |     !(function renderControls() {
410 |         (modeButtons.forEach((button) => {
411 |             const isActive := button.dataset.mode == mode;
412 |             (button.classList.toggle('is-active', isActive),
413 |             button.setAttribute('aria-pressed', String(isActive)));
414 |         })),
415 |         colorButtons.forEach((button) => {
416 |             const isActive := button.dataset.color == humanColor;
417 |             (button.classList.toggle('is-active', isActive),

```

```

418 |         button.setAttribute('aria-pressed', String(isActive));
419 |     },
420 |     (colorControlsElement.hidden = 'ai' !== mode),
421 |     (aiControlsElement.hidden = 'ai' !== mode),
422 |     (function renderAiSettingsControls() {
423 |         presetButtons.forEach((button) => {
424 |             const isActive = button.dataset.preset === aiSettings.preset;
425 |             (button.classList.toggle('is-active', isActive),
426 |             button.setAttribute('aria-pressed', String(isActive)));
427 |         }),
428 |         (aiDepthInput.value =
429 |         'hard' === aiSettings.preset ? aiDepthInput.max : String(aiSettings.maxDepth)),
430 |         (aiDepthInput.disabled = 'hard' === aiSettings.preset),
431 |         (aiDepthValue.textContent =
432 |         'hard' === aiSettings.preset ? '무제한' : `${aiSettings.maxDepth}`),
433 |         (aiTimeInput.value = String(aiSettings.timeLimitMs)),
434 |         (aiTimeInput.disabled = 'hard' === aiSettings.preset),
435 |         (aiTimeValue.textContent =
436 |         'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
437 |     })(),
438 |     (function renderClockSettingsControls() {
439 |         clockPresetButtons.forEach((button) => {
440 |             const isActive = button.dataset.clockPreset === clockPreset;
441 |             (button.classList.toggle('is-active', isActive),
442 |             button.setAttribute('aria-pressed', String(isActive)));
443 |         });
444 |     })(),
445 |     (undoButton.disabled = 0 === game.history().length));
446 | })(),
447 | renderClocks(),
448 | (function renderBoard() {
449 |     const fragment = document.createDocumentFragment(),
450 |     orientation = getBoardOrientation(),
451 |     files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
452 |     ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
453 |     lastMove = (function getLastMove() {
454 |         return game.history({ verbose: !0 }).at(-1) ?? null;
455 |     })();
456 |     boardElement.classList.toggle('is-locked', !canUseBoard());
457 |     for (const rank of ranks)
458 |         for (const file of files) {
459 |             const square = toSquare(file, rank),
460 |             piece = game.get(square),
461 |             button = document.createElement('button');
462 |             if (
463 |                 ((button.type = 'button'),
464 |                 (button.className = getSquareClasses(square, lastMove)),
465 |                 (button.dataset.square = square),
466 |                 (button.disabled = !canUseBoard()),
467 |                 button.setAttribute('role', 'gridcell'),
468 |                 button.setAttribute('aria-label', getSquareLabel(square, piece)),
469 |                 file === files[0] && button.appendChild(createCoordinate('rank', String(rank))),
470 |                 rank === ranks[ranks.length - 1] &&
471 |                 button.appendChild(createCoordinate('file', file)),
472 |                 piece)
473 |             ) {
474 |                 const pieceElement = document.createElement('span');
475 |                 ((pieceElement.className = 'piece'),
476 |                 (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
477 |                 button.appendChild(pieceElement));
478 |             }
479 |             fragment.appendChild(button);
480 |         }
481 |     boardElement.replaceChildren(fragment);
482 | })(),
483 | renderStatus(),
484 | renderAiCandidateDebug(),
485 | (function renderMoveList() {
486 |     const history = game.history({ verbose: !0 }),
487 |     fragment = document.createDocumentFragment();
488 |     if (0 === history.length) {

```



```

489 |         const empty = document.createElement('p');
490 |         return (
491 |             (empty.className = 'move-empty'),
492 |             (empty.textContent = '아직 돈 수가 없습니다.'),
493 |             fragment.appendChild(empty),
494 |             void moveListElement.replaceChildren(fragment)
495 |         );
496 |     }
497 |     for (let index = 0; index < history.length; index += 2) {
498 |         const row = document.createElement('div'),
499 |             moveNumber = document.createElement('span'),
500 |             whiteMove = document.createElement('span'),
501 |             blackMove = document.createElement('span');
502 |         ((row.className = 'move-row'),
503 |         (moveNumber.className = 'move-number'),
504 |         (whiteMove.className = 'move-san'),
505 |         (blackMove.className = 'move-san'),
506 |         (moveNumber.textContent = `${Math.floor(index / 2) + 1}`),
507 |         (whiteMove.textContent = history[index]?.san ?? ''),
508 |         (blackMove.textContent = history[index + 1]?.san ?? '')),
509 |         row.append(moveNumber, whiteMove, blackMove),
510 |         fragment.appendChild(row));
511 |     }
512 |     (moveListElement.replaceChildren(fragment),
513 |     (moveListElement.scrollTop = moveListElement.scrollHeight));
514 |     })(),
515 |     (function renderPromotionDialog() {
516 |         if (
517 |             ((promotionOverlay.hidden = null === pendingPromotion),
518 |             promotionChoices.replaceChildren(),
519 |             pendingPromotion)
520 |         )
521 |             for (const piece of PROMOTION_PIECES) {
522 |                 const button = document.createElement('button');
523 |                 ((button.className = 'promotion-button'),
524 |                 (button.type = 'button'),
525 |                 (button.dataset.promotion = piece),
526 |                 button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
527 |                 (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
528 |                 promotionChoices.appendChild(button));
529 |             }
530 |         })());
531 | }
532 | function renderClocks() {
533 |     const bottomColor = getBoardOrientation(),
534 |         topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
535 |         bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
536 |     ((clockDisplayElement.children[0] === topClockElement &&
537 |     clockDisplayElement.children[1] === bottomClockElement) ||
538 |     clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
539 |     renderClockPosition('w', bottomColor),
540 |     renderClockPosition('b', bottomColor),
541 |     renderClock('w', whiteClockElement, whiteClockTimeElement),
542 |     renderClock('b', blackClockElement, blackClockTimeElement));
543 | }
544 | function renderClockPosition(color, bottomColor) {
545 |     const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
546 |         isBottom = color === bottomColor;
547 |     (clockElement.classList.toggle('is-bottom', isBottom),
548 |     clockElement.classList.toggle('is-top', !isBottom));
549 | }
550 | function renderClock(color, clockElement, timeElement) {
551 |     const remainingMs = clockMs[color],
552 |         isActive = activeClockColor === color && null !== timedOutColor,
553 |         isTimedOut = timedOutColor === color;
554 |     (clockElement.classList.toggle('is-active', isActive),
555 |     clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
556 |     clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
557 |     clockElement.classList.toggle('is-timeout', isTimedOut),
558 |     (timeElement.textContent = (function formatClockTime(millisecons) {
559 |         const safeMilliseconds = Math.max(0, millisecons);

```

```

560 |         if (safeMilliseconds < 1e4)
561 |             return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
562 |         const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
563 |         return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
564 |     })(remainingMs));
565 | }
566 | function renderStatus() {
567 |     const historyLength = game.history().length;
568 |     ((statusElement.textContent = (function getStatusText() {
569 |         return game.isCheckmate()
570 |             ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
571 |             : game.isStalemate()
572 |             ? '스테일메이트 무승부'
573 |             : game.isInsufficientMaterial()
574 |             ? '기물 부족 무승부'
575 |             : game.isThreefoldRepetition()
576 |             ? '3회 반복 무승부'
577 |             : game.isDrawByFiftyMoves()
578 |             ? '50수 규칙 무승부'
579 |             : game.isDraw()
580 |             ? '무승부'
581 |             : timedOutColor
582 |             ? [
583 |                 `${colorName(timedOutColor)} 시간 초과`,
584 |                 `${colorName(oppositeColor(timedOutColor))} 승리`,
585 |             ].join('...')
586 |             : aiThinking
587 |             ? 'AI 계산 중'
588 |             : game.isCheck()
589 |             ? `${colorName(game.turn())} 체크`
590 |             : `${colorName(game.turn())} 차례`;
591 |     })()),
592 |     (turnMetaElement.textContent = colorName(game.turn())),
593 |     (modeMetaElement.textContent =
594 |         'local' === mode
595 |         ? '로컬 2인'
596 |         : ['AI 상태', `인간 ${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
597 |             '...',
598 |         )),
599 |     (moveMetaElement.textContent = [`${game.moveNumber()}수`, `${historyLength} half-move`].join(
600 |         '...',
601 |     )));
602 | }
603 | function renderAiCandidateDebug() {
604 |     if ('ai' !== mode)
605 |         return (
606 |             (aiCandidatePanelElement.hidden = !0),
607 |             (aiCandidateMetaElement.textContent = ''),
608 |             (aiInfoStatusElement.textContent = ''),
609 |             (aiInfoPresetElement.textContent = ''),
610 |             (aiInfoHumanElement.textContent = ''),
611 |             (aiInfoDepthElement.textContent = ''),
612 |             (aiInfoNodesElement.textContent = ''),
613 |             (aiInfoScoreElement.textContent = ''),
614 |             void aiCandidateListElement.replaceChildren()
615 |         );
616 |     const debug = aiProgress?.debug;
617 |     if (
618 |         ((aiCandidatePanelElement.hidden = !1),
619 |         (aiCandidateMetaElement.textContent = aiProgress
620 |             ? `경과 ${formatSecnds(aiProgress.elapsedMs)}`
621 |             : '')),
622 |         (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
623 |             return game.isGameOver() || null !== timedOutColor
624 |             ? '대국 종료'
625 |             : aiThinking
626 |             ? aiProgress
627 |             ? '계산 중'
628 |             : '계산 준비 중'
629 |             : game.turn() === humanColor
630 |             ? '대기 중'

```

```

631 |         : '응수·대기';
632 |     }()),
633 |     (aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
634 |     (aiInfoHumanElement.textContent = colorName(humanColor)),
635 |     (aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
636 |     (aiInfoNodesElement.textContent = aiProgress
637 |         ? ` ${ (function formatNodes(nodes) {
638 |             return nodes ≥ 1e6 ? `${Math.round(nodes / 1e3)}k` : String(nodes);
639 |         })(aiProgress.nodes) } ` nodes
640 |         : '-'),
641 |     (aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
642 |     !debug || 0 === debug.candidates.length
643 |     )
644 |     return void aiCandidateListElement.replaceChildren();
645 |     aiCandidateMetaElement.textContent = [
646 |         aiCandidateMetaElement.textContent,
647 |         `best·${formatDebugScore(debug.bestScore)}`,
648 |         `창·${Math.round(debug.windowCp)}`,
649 |         `p·${debug.ticketPower.toFixed(2)}`,
650 |     ].join('·');
651 |     const fragment = document.createDocumentFragment();
652 |     for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
653 |     aiCandidateListElement.replaceChildren(fragment);
654 | }
655 | function createAiCandidateRow(candidate) {
656 |     const row = document.createElement('div');
657 |     ((row.className = 'ai-candidate-row'),
658 |     row.classList.toggle('is-selected', candidate.selected),
659 |     row.classList.toggle('is-principal', candidate.principal),
660 |     row.setAttribute('role', 'row'));
661 |     const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
662 |     return (
663 |     candidate.selected && sanCell.appendChild(createAiCandidateBadge('선택')),
664 |     candidate.principal && sanCell.appendChild(createAiCandidateBadge('최선')),
665 |     row.append(
666 |     sanCell,
667 |     createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
668 |     createAiCandidateCell(
669 |     'ai-candidate-weight',
670 |     (function formatDebugWeight(weight) {
671 |         return 0 === weight
672 |         ? '0'
673 |         : weight < 0.001
674 |         ? '<0.001'
675 |         : weight.toFixed(3).replace(/0+$/, '').replace(/\./, '');
676 |     })(candidate.tickets),
677 |     ),
678 |     createAiCandidateCell(
679 |     'ai-candidate-probability',
680 |     (function formatDebugProbability(probability) {
681 |         const percent = 100 * probability;
682 |         return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
683 |     })(candidate.probability),
684 |     ),
685 |     ),
686 |     row
687 | );
688 | }
689 | function createAiCandidateCell(className, text) {
690 |     const cell = document.createElement('span');
691 |     return (
692 |     (cell.className = className),
693 |     cell.setAttribute('role', 'cell'),
694 |     (cell.textContent = text),
695 |     cell
696 | );
697 | }
698 | function createAiCandidateBadge(text) {
699 |     const badge = document.createElement('span');
700 |     return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
701 | }

```

```

702 |     function getSquareClasses(square, lastMove) {
703 |         const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
704 |         destinationMove = legalMoves.find((move) => move.to === square);
705 |         return (
706 |             selectedSquare === square && classes.push('is-selected'),
707 |             destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
708 |             !lastMove ||
709 |             (lastMove.from !== square && lastMove.to !== square) ||
710 |             classes.push('is-last-move'),
711 |             (function isCheckedKing(square) {
712 |                 const piece = game.get(square);
713 |                 return Boolean(
714 |                     piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
715 |                     );
716 |             })(square) && classes.push('is-check'),
717 |             classes.join(' ')
718 |         );
719 |     }
720 |     function getBoardOrientation() {
721 |         return 'ai' === mode ? humanColor : game.turn();
722 |     }
723 |     function canUseBoard() {
724 |         return !(
725 |             game.isGameOver() ||
726 |             null !== timedOutColor ||
727 |             aiThinking ||
728 |             pendingPromotion ||
729 |             ('local' !== mode && game.turn() !== humanColor)
730 |         );
731 |     }
732 |     (boardElement.addEventListener('click', (event) => {
733 |         const button = (function getSquareButton(target, boardElement) {
734 |             if (!target instanceof Element) return null;
735 |             const button = target.closest('.board-square');
736 |             return button && boardElement.contains(button) ? button : null;
737 |         })(event.target, boardElement);
738 |         button &&
739 |         (function isSquare(value) {
740 |             return 'string' === typeof value && SQUARE_SET.has(value);
741 |         })(button.dataset.square) &&
742 |         (function handleSquareClick(square) {
743 |             if (!canUseBoard()) return;
744 |             const piece = game.get(square);
745 |             if (selectedSquare) {
746 |                 const movesToDestination = legalMoves.filter((move) => move.to === square);
747 |                 return movesToDestination.length > 0
748 |                     ? movesToDestination.some((move) => move.isPromotion())
749 |                     ? ((pendingPromotion = {
750 |                         from: selectedSquare,
751 |                         to: square,
752 |                         moves: movesToDestination,
753 |                     }),
754 |                       void render())
755 |                     : void playMove({ from: selectedSquare, to: square })
756 |                     : piece?.color === game.turn()
757 |                     ? void selectSquare(square)
758 |                     : (clearSelection(), void render());
759 |             }
760 |             piece?.color === game.turn() && selectSquare(square);
761 |         })(button.dataset.square);
762 |     }),
763 |     modeButtons.forEach((button) => {
764 |         button.addEventListener('click', () => {
765 |             const nextMode = button.dataset.mode;
766 |             (function isGameMode(value) {
767 |                 return 'local' === value || 'ai' === value;
768 |             })(nextMode) &&
769 |             nextMode !== mode &&
770 |             ((mode = nextMode), resetGame());
771 |         });
772 |     }),

```

```

773 |     colorButtons.forEach((button) => {
774 |         button.addEventListener('click', () => {
775 |             const nextColor = button.dataset.color;
776 |             (function isColor(value) {
777 |                 return 'w' === value || 'b' === value;
778 |             })(nextColor) &&
779 |                 nextColor !== humanColor &&
780 |                 ((humanColor = nextColor), resetGame());
781 |         });
782 |     }),
783 |     presetButtons.forEach((button) => {
784 |         button.addEventListener('click', () => {
785 |             const preset = button.dataset.preset;
786 |             (function isAiPreset(value) {
787 |                 return 'easy' === value || 'normal' === value || 'hard' === value;
788 |             })(preset) && updateAiSettings({ ... AI_PRESETS[preset] });
789 |         });
790 |     }),
791 |     clockPresetButtons.forEach((button) => {
792 |         button.addEventListener('click', () => {
793 |             const preset = button.dataset.clockPreset;
794 |             (function isClockPreset(value) {
795 |                 return (
796 |                     '1/1' === value ||
797 |                     '3/2' === value ||
798 |                     '5/3' === value ||
799 |                     '10/5' === value ||
800 |                     '15/10' === value
801 |                 );
802 |             })(preset) &&
803 |                 preset !== clockPreset &&
804 |                 ((clockPreset = preset), resetGame());
805 |         });
806 |     }),
807 |     aiDepthInput.addEventListener('input', () => {
808 |         var value;
809 |         updateAiSettings({
810 |             ... aiSettings,
811 |             maxDepth:
812 |                 ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
813 |         });
814 |     }),
815 |     aiTimeInput.addEventListener('input', () => {
816 |         updateAiSettings({ ... aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
817 |     }),
818 |     newGameButton.addEventListener('click', resetGame),
819 |     undoButton.addEventListener('click', function undoLastTurn() {
820 |         (cancelAiMove(),
821 |         (pendingPromotion = null),
822 |         0 !== game.history().length
823 |         ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
824 |         clearSelection(),
825 |         resetClockState(),
826 |         restartClockForCurrentTurn(),
827 |         render())
828 |         : render());
829 |     }),
830 |     promotionChoices.addEventListener('click', (event) => {
831 |         const target = event.target;
832 |         if (!(target instanceof Element)) return;
833 |         const button = target.closest('[data-promotion]'),
834 |             promotion = button ? dataset.promotion :
835 |             (function isPromotionPiece(value) {
836 |                 return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
837 |             })(promotion) &&
838 |             (function finishPromotion(promotion) {
839 |                 if (!pendingPromotion) return;
840 |                 if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
841 |                 const { from, to } = pendingPromotion;
842 |                 ((pendingPromotion = null), playMove({ from, to, promotion }));
843 |             })(promotion);

```

```

844 |     },
845 |     resetClockState(),
846 |     restartClockForCurrentTurn(),
847 |     render());
848 |   }());
849 |

```

==== style.css (14083 B / 3bb92984b4f4) ====

```

1 | *,
2 | *:before,
3 | *:after {
4 |   box-sizing: border-box;
5 | }
6 | html {
7 |   min-width: 1280px;
8 |   min-height: 100%;
9 |   background: var(--page);
10 |  color-scheme: light;
11 | }
12 | body {
13 |   min-width: 1280px;
14 |   min-height: 100vh;
15 |   min-height: 100dvh;
16 |   margin: 0;
17 |   background: var(--page);
18 |   font-family:
19 |     Inter,
20 |     ui-sans-serif,
21 |     system-ui,
22 |     -apple-system,
23 |     BlinkMacSystemFont,
24 |     Segoe UI,
25 |     sans-serif;
26 |   text-rendering: geometricPrecision;
27 | }
28 | button {
29 |   font: inherit;
30 | }
31 | button:not(:disabled) {
32 |   cursor: pointer;
33 | }
34 | button:disabled {
35 |   cursor: not-allowed;
36 | }
37 | :root {
38 |   --app-padding: 28px;
39 |   --viewport-height: 100vh;
40 |   --page: oklch(95% 0.02 85deg / 1);
41 |   --ink: oklch(26% 0.01 250deg / 1);
42 |   --muted: oklch(54% 0.01 185deg / 1);
43 |   --line: oklch(81% 0.02 80deg / 1);
44 |   --panel: oklch(99% 0.01 85deg / 1);
45 |   --button: oklch(33% 0.03 205deg / 1);
46 |   --button-text: oklch(100% 0 0deg / 1);
47 |   --accent: oklch(55% 0.15 35deg / 1);
48 |   --light-square: oklch(89% 0.05 90deg / 1);
49 |   --dark-square: oklch(59% 0.08 150deg / 1);
50 |   --last-move: oklch(80% 0.12 90deg / 1);
51 |   --legal: oklch(43% 0.12 260deg / 1);
52 |   --capture: oklch(52% 0.15 25deg / 1);
53 |   --shadow: 0 18px 42px oklch(24% 0.01 75deg / 0.18);
54 |   --clear-color: oklch(0% 0 0deg / 0);
55 |   --panel-soft: oklch(99% 0.01 85deg / 0.68);
56 |   --panel-raised: oklch(99% 0.01 85deg / 0.74);
57 |   --panel-strong: oklch(99% 0.01 85deg / 0.88);
58 |   --board-edge: oklch(33% 0.03 155deg / 1);
59 |   --last-move-strong: oklch(80% 0.12 90deg / 0.76);
60 |   --last-move-soft: oklch(80% 0.12 90deg / 0.68);
61 |   --legal-marker: oklch(43% 0.12 260deg / 0.42);
62 |   --capture-marker: oklch(52% 0.15 25deg / 0.74);
63 |   --piece-shadow: oklch(0% 0 0deg / 0.2);

```

```

64 | --active-ring: oklch(100% 0 0deg / 0.3);
65 | --active-halo: oklch(33% 0.03 205deg / 0.18);
66 | --warning: oklch(69% 0.15 75deg / 1);
67 | --line-soft: oklch(81% 0.02 80deg / 0.58);
68 | --line-firm: oklch(81% 0.02 80deg / 0.62);
69 | --accent-soft: oklch(55% 0.15 35deg / 0.12);
70 | --button-soft: oklch(33% 0.03 205deg / 0.08);
71 | --overlay: oklch(21% 0.01 240deg / 0.58);
72 | }
73 | @supports (height: 100dvh) {
74 |   :root {
75 |     --viewport-height: 100dvh;
76 |   }
77 | }
78 | [hidden] {
79 |   display: none !important;
80 | }
81 | .chess-app {
82 |   --board-min-size: 560px;
83 |   --board-vertical-breathing: 104px;
84 |   --side-columns-width: 628px;
85 |   --grid-gaps-width: 30px;
86 |   --board-size: max(
87 |     var(--board-min-size),
88 |     min(
89 |       calc(
90 |         var(--viewport-height) - var(--app-padding) - var(--app-padding) -
91 |         var(--board-vertical-breathing)
92 |       ),
93 |       calc(
94 |         100vw - var(--app-padding) - var(--app-padding) - var(--side-columns-width) -
95 |         var(--grid-gaps-width)
96 |       )
97 |     )
98 |   );
99 |   --panel-max-height: calc(var(--viewport-height) - var(--app-padding) - var(--app-padding));
100 |   width: 100%;
101 |   min-width: 1280px;
102 |   min-height: var(--viewport-height);
103 |   display: grid;
104 |   grid-template-columns: 230px var(--board-size) 118px 280px;
105 |   gap: 10px;
106 |   align-items: stretch;
107 |   justify-content: center;
108 |   overflow: hidden;
109 |   padding: var(--app-padding);
110 |   background: var(--page);
111 |   color: var(--ink);
112 | }
113 | .board-stage {
114 |   width: var(--board-size);
115 |   align-self: center;
116 |   justify-self: center;
117 | }
118 | .clock-panel {
119 |   width: 100%;
120 |   height: var(--board-size);
121 |   min-width: 0;
122 |   min-height: 0;
123 |   align-self: center;
124 |   justify-self: stretch;
125 |   display: flex;
126 | }
127 | .chess-board {
128 |   width: 100%;
129 |   aspect-ratio: 1;
130 |   display: grid;
131 |   grid-template-columns: repeat(8, minmax(0, 1fr));
132 |   overflow: hidden;
133 |   border: 3px solid var(--board-edge);
134 |   border-radius: 8px;

```

```

135 |   box-shadow: var(--shadow);
136 |   background: var(--board-edge);
137 | }
138 | .board-square {
139 |   position: relative;
140 |   width: 100%;
141 |   aspect-ratio: 1;
142 |   display: grid;
143 |   place-items: center;
144 |   min-width: 0;
145 |   padding: 0;
146 |   border: 0;
147 |   color: var(--ink);
148 |   transition:
149 |     box-shadow 0.14s ease,
150 |     filter 0.14s ease,
151 |     transform 0.14s ease;
152 | }
153 | .board-square:not(:disabled):hover {
154 |   filter: brightness(1.05);
155 | }
156 | .board-square.is-light {
157 |   background: var(--light-square);
158 | }
159 | .board-square.is-dark {
160 |   background: var(--dark-square);
161 | }
162 | .board-square.is-selected {
163 |   box-shadow: inset 0 0 0 4px var(--legal);
164 | }
165 | .board-square.is-last-move {
166 |   box-shadow: inset 0 0 0 4px var(--last-move-strong);
167 | }
168 | .board-square.is-selected.is-last-move {
169 |   box-shadow:
170 |     inset 0 0 0 4px var(--legal),
171 |     inset 0 0 0 8px var(--last-move-soft);
172 | }
173 | .board-square.is-legal:after,
174 | .board-square.is-capture:after {
175 |   position: absolute;
176 |   z-index: 1;
177 |   content: '';
178 |   border-radius: 999px;
179 |   pointer-events: none;
180 | }
181 | .board-square.is-legal:after {
182 |   width: 28%;
183 |   height: 28%;
184 |   background: var(--legal-marker);
185 | }
186 | .board-square.is-capture:after {
187 |   inset: 12%;
188 |   border: 3px solid var(--capture-marker);
189 | }
190 | .board-square.is-check {
191 |   box-shadow: inset 0 0 0 5px var(--capture);
192 | }
193 | .chess-board.is-locked .board-square {
194 |   filter: saturate(0.86);
195 | }
196 | .piece {
197 |   z-index: 2;
198 |   display: block;
199 |   font-size: 4rem;
200 |   font-size: max(4rem, var(--board-size) / 8 * 0.92);
201 |   line-height: 1;
202 |   text-shadow: 0 2px 6px var(--piece-shadow);
203 |   -webkit-user-select: none;
204 |   user-select: none;
205 | }

```



```

206 | .coordinate-{
207 |   position: absolute;
208 |   z-index: 3;
209 |   font-size: 0.7rem;
210 |   font-size: max(0.7rem, var(--board-size) / 80);
211 |   font-weight: 800;
212 |   line-height: 1;
213 |   opacity: 0.72;
214 |   pointer-events: none;
215 | }
216 | .coordinate-rank-{
217 |   top: 6px;
218 |   left: 7px;
219 | }
220 | .coordinate-file-{
221 |   right: 7px;
222 |   bottom: 6px;
223 | }
224 | .control-panel,
225 | .info-panel-{
226 |   min-width: 0;
227 |   min-height: 0;
228 |   max-height: var(--panel-max-height);
229 |   display: flex;
230 |   flex-direction: column;
231 |   gap: 14px;
232 |   overflow: auto;
233 |   overscroll-behavior: contain;
234 |   scrollbar-gutter: stable;
235 | }
236 | .control-panel-{
237 |   justify-self: end;
238 | }
239 | .info-panel-{
240 |   justify-self: start;
241 | }
242 | .control-label-{
243 |   margin: 0;
244 |   color: var(--muted);
245 |   font-size: 0.78rem;
246 |   font-weight: 800;
247 |   letter-spacing: 0;
248 |   text-transform: uppercase;
249 | }
250 | .control-group,
251 | .status-panel,
252 | .ai-candidate-panel,
253 | .history-panel-{
254 |   display: grid;
255 |   gap: 10px;
256 |   padding-top: 16px;
257 |   border-top: 1px solid var(--line);
258 | }
259 | .history-panel-{
260 |   min-height: 0;
261 |   flex: 1;
262 |   grid-template-rows: auto minmax(0, 1fr);
263 | }
264 | .segmented-control-{
265 |   display: grid;
266 |   grid-template-columns: repeat(2, minmax(0, 1fr));
267 |   gap: 4px;
268 |   padding: 4px;
269 |   border: 1px solid var(--line);
270 |   border-radius: 8px;
271 |   background: var(--panel-soft);
272 | }
273 | .segmented-control-three-{
274 |   grid-template-columns: repeat(3, minmax(0, 1fr));
275 | }
276 | .clock-preset-control-{

```

```

277 |   display: grid;
278 |   grid-template-columns: repeat(5, minmax(0, 1fr));
279 |   gap: 4px;
280 |   padding: 4px;
281 |   border: 1px solid var(--line);
282 |   border-radius: 8px;
283 |   background: var(--panel-soft);
284 | }
285 | .segmented-button,
286 | .primary-button,
287 | .ghost-button,
288 | .promotion-button {
289 |   min-height: 42px;
290 |   border-radius: 6px;
291 |   font-weight: 800;
292 |   letter-spacing: 0;
293 | }
294 | .segmented-button {
295 |   border: 0;
296 |   background: var(--clear-color);
297 |   color: var(--muted);
298 | }
299 | .segmented-button.is-active {
300 |   background: var(--button);
301 |   color: var(--button-text);
302 | }
303 | .clock-settings .segmented-button {
304 |   min-height: 36px;
305 |   padding-inline: 4px;
306 |   font-size: 0.82rem;
307 | }
308 | .ai-settings {
309 |   gap: 12px;
310 | }
311 | .range-control,
312 | .toggle-control {
313 |   display: grid;
314 |   gap: 8px;
315 |   min-width: 0;
316 |   color: var(--ink);
317 |   font-size: 0.92rem;
318 |   font-weight: 800;
319 | }
320 | .range-control span,
321 | .toggle-control {
322 |   align-items: center;
323 | }
324 | .range-control span {
325 |   display: flex;
326 |   justify-content: space-between;
327 |   gap: 10px;
328 | }
329 | .range-control strong {
330 |   color: var(--accent);
331 |   white-space: nowrap;
332 | }
333 | .range-control input[type='range'] {
334 |   width: 100%;
335 |   accent-color: var(--button);
336 | }
337 | .toggle-control {
338 |   grid-template-columns: auto minmax(0, 1fr);
339 |   padding: 10px;
340 |   border: 1px solid var(--line);
341 |   border-radius: 8px;
342 |   background: var(--panel-raised);
343 | }
344 | .toggle-control input {
345 |   width: 18px;
346 |   height: 18px;
347 |   accent-color: var(--button);

```

```

348 | }
349 | .action-row {
350 |   display: grid;
351 |   grid-template-columns: 1fr 1fr;
352 |   gap: 10px;
353 | }
354 | .primary-button,
355 | .ghost-button {
356 |   border: 1px solid var(--button);
357 | }
358 | .primary-button {
359 |   background: var(--button);
360 |   color: var(--button-text);
361 | }
362 | .ghost-button {
363 |   background: var(--clear-color);
364 |   color: var(--button);
365 | }
366 | .primary-button:disabled,
367 | .ghost-button:disabled,
368 | .segmented-button:disabled {
369 |   opacity: 0.46;
370 | }
371 | .game-status {
372 |   min-height: 1.4em;
373 |   color: var(--accent);
374 |   font-size: 1.35rem;
375 |   line-height: 1.2;
376 | }
377 | .clock-display {
378 |   width: 100%;
379 |   height: 100%;
380 |   min-height: 0;
381 |   min-width: 0;
382 |   display: flex;
383 |   flex-direction: column;
384 |   gap: 10px;
385 |   justify-content: space-between;
386 | }
387 | .clock-card {
388 |   --clock-bg: var(--panel-raised);
389 |   --clock-border: var(--line);
390 |   --clock-side: var(--muted);
391 |   --clock-time: var(--ink);
392 |   width: 100%;
393 |   min-block-size: 82px;
394 |   max-block-size: 110px;
395 |   min-width: 0;
396 |   display: grid;
397 |   place-content: center;
398 |   gap: 4px;
399 |   padding: 12px 8px;
400 |   border: 1px solid var(--clock-border);
401 |   border-radius: 8px;
402 |   background: var(--clock-bg);
403 |   text-align: center;
404 | }
405 | #white-clock {
406 |   --clock-bg: oklch(99% 0.01 85deg / 1);
407 |   --clock-border: oklch(81% 0.02 80deg / 1);
408 |   --clock-side: oklch(54% 0.01 185deg / 1);
409 |   --clock-time: oklch(26% 0.01 250deg / 1);
410 | }
411 | #black-clock {
412 |   --clock-bg: oklch(33% 0.03 205deg / 1);
413 |   --clock-border: oklch(25% 0.02 210deg / 1);
414 |   --clock-side: oklch(99% 0.01 85deg / 0.74);
415 |   --clock-time: oklch(99% 0.01 85deg / 1);
416 | }
417 | .clock-card.is-active {
418 |   border-color: var(--button);

```

```

419 |   box-shadow:
420 |     inset 0 0 0 2px var(--active-ring),
421 |     0 0 0 2px var(--active-halo);
422 | }
423 | .clock-card.is-warning .clock-time {
424 |   color: var(--warning);
425 | }
426 | .clock-card.is-danger .clock-time,
427 | .clock-card.is-timeout .clock-time {
428 |   color: var(--capture);
429 | }
430 | .clock-side {
431 |   color: var(--clock-side);
432 |   font-size: 0.72rem;
433 |   font-weight: 800;
434 |   line-height: 1;
435 | }
436 | .clock-time {
437 |   font-variant-numeric: tabular-nums;
438 |   color: var(--clock-time);
439 |   font-size: 1.35rem;
440 |   line-height: 1;
441 | }
442 | .status-grid {
443 |   display: grid;
444 |   grid-template-columns: 1fr;
445 |   gap: 8px;
446 |   margin: 0;
447 | }
448 | .status-grid div {
449 |   min-width: 0;
450 |   padding: 10px;
451 |   border: 1px solid var(--line);
452 |   border-radius: 8px;
453 |   background: var(--panel-raised);
454 | }
455 | .status-grid dt,
456 | .status-grid dd {
457 |   margin: 0;
458 | }
459 | .status-grid dt {
460 |   color: var(--muted);
461 |   font-size: 0.72rem;
462 |   font-weight: 800;
463 | }
464 | .status-grid dd {
465 |   margin-top: 4px;
466 |   overflow-wrap: anywhere;
467 |   font-size: 0.92rem;
468 |   font-weight: 800;
469 | }
470 | .history-header {
471 |   display: flex;
472 |   align-items: center;
473 |   justify-content: space-between;
474 | }
475 | .ai-candidate-panel {
476 |   min-height: 0;
477 | }
478 | .ai-info-grid {
479 |   display: grid;
480 |   grid-template-columns: repeat(2, minmax(0, 1fr));
481 |   gap: 6px;
482 |   margin: 0;
483 | }
484 | .ai-info-grid div {
485 |   min-width: 0;
486 |   padding: 8px;
487 |   border: 1px solid var(--line);
488 |   border-radius: 8px;
489 |   background: var(--panel-raised);

```

```

490 | }
491 | .ai-info-grid.dt,
492 | .ai-info-grid.dd {
493 |   margin: 0;
494 | }
495 | .ai-info-grid.dt {
496 |   color: var(--muted);
497 |   font-size: 0.64rem;
498 |   font-weight: 800;
499 |   text-transform: uppercase;
500 | }
501 | .ai-info-grid.dd {
502 |   margin-top: 3px;
503 |   overflow: hidden;
504 |   font-size: 0.8rem;
505 |   font-weight: 800;
506 |   line-height: 1.1;
507 |   text-overflow: ellipsis;
508 |   white-space: nowrap;
509 | }
510 | .ai-info-grid.dd[id='ai-info-nodes'],
511 | .ai-info-grid.dd[id='ai-info-score'],
512 | .ai-info-grid.dd[id='ai-info-depth'] {
513 |   font-variant-numeric: tabular-nums;
514 | }
515 | .ai-candidate-header {
516 |   display: flex;
517 |   align-items: baseline;
518 |   justify-content: space-between;
519 |   gap: 8px;
520 | }
521 | .ai-candidate-meta {
522 |   min-width: 0;
523 |   overflow: hidden;
524 |   color: var(--muted);
525 |   font-size: 0.68rem;
526 |   font-weight: 800;
527 |   text-align: right;
528 |   text-overflow: ellipsis;
529 |   white-space: nowrap;
530 | }
531 | .ai-candidate-table {
532 |   min-height: 0;
533 |   overflow: hidden;
534 |   border: 1px solid var(--line);
535 |   border-radius: 8px;
536 |   background: var(--panel-raised);
537 | }
538 | .ai-candidate-list {
539 |   max-height: 172px;
540 |   overflow: auto;
541 | }
542 | .ai-candidate-row {
543 |   display: grid;
544 |   grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
545 |   gap: 6px;
546 |   align-items: center;
547 |   min-width: 0;
548 |   padding: 6px 8px;
549 |   border-bottom: 1px solid var(--line-soft);
550 |   font-size: 0.72rem;
551 |   font-weight: 800;
552 |   line-height: 1.15;
553 | }
554 | .ai-candidate-row:last-child {
555 |   border-bottom: 0;
556 | }
557 | .ai-candidate-head {
558 |   color: var(--muted);
559 |   background: var(--panel-strong);
560 |   font-size: 0.66rem;

```

```

561 |   text-transform: uppercase;
562 | }
563 | .ai-candidate-row.is-selected {
564 |   background: var(--accent-soft);
565 | }
566 | .ai-candidate-row.is-principal:not(.is-selected) {
567 |   background: var(--button-soft);
568 | }
569 | .ai-candidate-san,
570 | .ai-candidate-score,
571 | .ai-candidate-weight,
572 | .ai-candidate-probability {
573 |   min-width: 0;
574 |   overflow: hidden;
575 |   text-overflow: ellipsis;
576 |   white-space: nowrap;
577 | }
578 | .ai-candidate-score,
579 | .ai-candidate-weight,
580 | .ai-candidate-probability {
581 |   text-align: right;
582 |   font-variant-numeric: tabular-nums;
583 | }
584 | .ai-candidate-badge {
585 |   display: inline-block;
586 |   margin-left: 4px;
587 |   padding: 1px 3px;
588 |   border-radius: 4px;
589 |   background: var(--button);
590 |   color: var(--button-text);
591 |   font-size: 0.56rem;
592 |   line-height: 1;
593 |   vertical-align: 1px;
594 | }
595 | .move-list {
596 |   min-height: 0;
597 |   max-height: none;
598 |   overflow: auto;
599 |   border: 1px solid var(--line);
600 |   border-radius: 8px;
601 |   background: var(--panel-raised);
602 | }
603 | .move-row {
604 |   display: grid;
605 |   grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
606 |   gap: 8px;
607 |   padding: 8px 10px;
608 |   border-bottom: 1px solid var(--line-firm);
609 |   font-weight: 800;
610 | }
611 | .move-row:last-child {
612 |   border-bottom: 0;
613 | }
614 | .move-number {
615 |   color: var(--muted);
616 | }
617 | .move-san {
618 |   min-width: 0;
619 |   overflow-wrap: anywhere;
620 | }
621 | .move-empty {
622 |   margin: 0;
623 |   padding: 14px 12px;
624 |   color: var(--muted);
625 |   font-weight: 700;
626 | }
627 | .promotion-overlay {
628 |   position: fixed;
629 |   inset: 0;
630 |   z-index: 10;
631 |   display: grid;

```

```
632 |     place-items: center;
633 |     padding: 20px;
634 |     background: var(--overlay);
635 | }
636 | .promotion-overlay[hidden] {
637 |     display: none;
638 | }
639 | .promotion-dialog {
640 |     width: 360px;
641 |     display: grid;
642 |     gap: 16px;
643 |     padding: 20px;
644 |     border: 1px solid var(--line);
645 |     border-radius: 8px;
646 |     background: var(--panel);
647 |     box-shadow: var(--shadow);
648 | }
649 | .promotion-dialog h2 {
650 |     margin: 0;
651 |     font-size: 1.1rem;
652 | }
653 | .promotion-choices {
654 |     display: grid;
655 |     grid-template-columns: repeat(4, minmax(0, 1fr));
656 |     gap: 8px;
657 | }
658 | .promotion-button {
659 |     min-height: 70px;
660 |     border: 1px solid var(--button);
661 |     background: var(--button-text);
662 |     color: var(--ink);
663 |     font-size: 2.4rem;
664 |     line-height: 1;
665 | }
666 |
```